FUNDAÇÃO DE APOIO À ESCOLA TÉCNICA DO RIO DE JANEIRO

ESCOLA TÉCNICA ESTADUAL SANTA CRUZ

THIAGO SOUZA ZOTTI

JONATHAN SAMUEL MARTINS TANNES

FaeHub+

Sistema de gestão escolar

Projeto final apresentado à Escola Técnica Estadual Santa Cruz como requisito parcial para obtenção do título de técnico em informática

Rio de Janeiro, 26 de novembro de 2026

Folha de apresentação e banca examinadora

Thiago Souza Zotti e Jonathan Samuel Martins Tannes

FaeHub+ — Sistema de gestão escolar

Projeto final apresentado à Escola Técnica Estadual Santa Cruz como requisito parcial para obtenção do título de Técnico em Informática.

Banca examinadora

_____

Professora Aline Silvestre Rosa Serrão — Orientadora

_____

Professora Luciana do Amaral Teixeira — Avaliadora

SUMÁRIO

1. DESCRIÇÃO DO CONTEXTO 4

2. DESCRIÇÃO DO SISTEMA 4

2.1. Requisitos do sistema 4

2.1.1. Requisitos funcionais 4

2.1.2. Requisitos não funcionais 5

2.1.3. Regras de negócio 6

3. CASOS DE USO 6

3.1. Diagrama de casos de uso 6

3.2. Especificação dos casos de uso 8

4. DIAGRAMA DE CLASSES 15

4.1. Especificação das classes 17

5. PROJETO DO BANCO DE DADOS 18

5.1. Modelo Conceitual: Modelo de entidade e relacionamento 18

5.2. Modelo físico 19

5.2.1. Diagrama físico do banco de dados 19

5.2.2. Script SQL de criação do banco de dados 20

6. MAPA DE NAVEGAÇÃO 31

7. MANUAL DO USUÁRIO 31

7.1. Acesso ao sistema 31

7.2. Apresentação das telas e funcionalidades 33

7.3. Instruções de utilização 36

1. DESCRIÇÃO DO CONTEXTO

O FaeHub+ é um sistema web de gestão escolar desenvolvido por Thiago Souza Zotti e Jonathan Samuel Martins Tannes no contexto da Escola Técnica Estadual Santa Cruz (ETESC / FAETEC).

O projeto reúne acompanhamento acadêmico, comunicação e serviços escolares. O aluno consulta seus registros. O professor trabalha com as turmas e disciplinas vinculadas. A direção administra as rotinas escolares e o responsável acompanha os estudantes vinculados à sua conta.

A proposta busca reduzir a dispersão de informações entre registros e canais diferentes. A integração com a FaeNet oferece um fluxo de acesso federado quando configurado.

2. DESCRIÇÃO DO SISTEMA

O servidor usa Python, Flask e templates Jinja2. A interface usa HTML, CSS e JavaScript. Os dados operacionais são persistidos no PostgreSQL do Supabase por meio de psycopg.

O servidor verifica a identidade, o estado da conta, o perfil e os vínculos acadêmicos antes de permitir ações protegidas. Convites e recuperação dependem de e-mail configurado. O autenticador acrescenta a segunda confirmação quando habilitado.

2.1. Requisitos do sistema

2.1.1. Requisitos funcionais

ID	Descrição
RF01	Manter contas com papéis escolares e vínculos, com convite e ativação de contas pessoais.
RF02	Lançar e consultar N1 e N2 por disciplina, calcular a média e apresentar a situação da consulta.
RF03	Registrar frequência por aula e matrícula, com disciplina, data, início e fim.
RF04	Consultar e manter horários por turma e oferta de disciplina, conforme permissão.
RF05	Consultar agenda e calendário escolar e manter eventos autorizados.
RF06	Registrar e acompanhar intervenções pedagógicas, sem nota comportamental automática.
RF07	Publicar exercícios com prazo, receber entregas e registrar devolutivas.
RF08	Publicar avisos para turmas, com prioridade e controle de publicação.
RF09	Publicar e consultar estágios, com prazo e orientações de candidatura.
RF10	Solicitar e atender documentos pela Secretaria, com protocolo e situação.
RF11	Oferecer mensagens, grupos e recursos de acompanhamento de conversa.
RF12	Registrar ações relevantes para auditoria, sem presumir registro de todas as interações.
RF13	Permitir envio, alteração e remoção da foto de perfil, com validação de arquivo.
RF14	Permitir acompanhamento de estudantes por responsáveis com vínculo ativo.
RF15	Manter anos, períodos, cursos, disciplinas, turmas, matrículas e vínculos docentes.
RF16	Consultar histórico acadêmico sem sobrescrever os vínculos de anos anteriores.
RF17	Manter diário de classe e permitir revisão conforme o fluxo escolar.
RF18	Oferecer recuperação por e-mail confirmado e autenticador opcional quando configurado.

2.1.2. Requisitos não funcionais

ID	Descrição
RNF01	As senhas devem usar hash de senha com salt e derivação de chave pelo Werkzeug. SHA-256 simples não é o mecanismo de armazenamento descrito.
RNF02	A interface deve ser responsiva e permitir navegação em dispositivos móveis e desktop, respeitando o perfil autorizado.
RNF03	O banco operacional deve ser PostgreSQL no Supabase, configurado no servidor por FAEHUB_DATABASE_URL ou DATABASE_URL.
RNF04	SQLite deve ficar restrito a testes isolados e migração legada, sem fallback operacional automático.
RNF05	O servidor deve validar conta ativa, papel e vínculos. Os quatro espaços principais são aluno, professor, direção e responsável.
RNF06	Publicação exige servidor WSGI, configuração de ambiente, HTTPS e backup separado. O envio ao GitHub não hospeda a aplicação.
RNF07	O ambiente deve oferecer Python 3.10 ou superior e as dependências de requirements.txt, além de conexão ao banco.
RNF08	Segredos e registros escolares não devem integrar repositório, documentação pública ou logs de depuração.
RNF09	Convites e recuperação dependem de e-mail configurado. MFA e federação dependem das respectivas chaves e configuração.

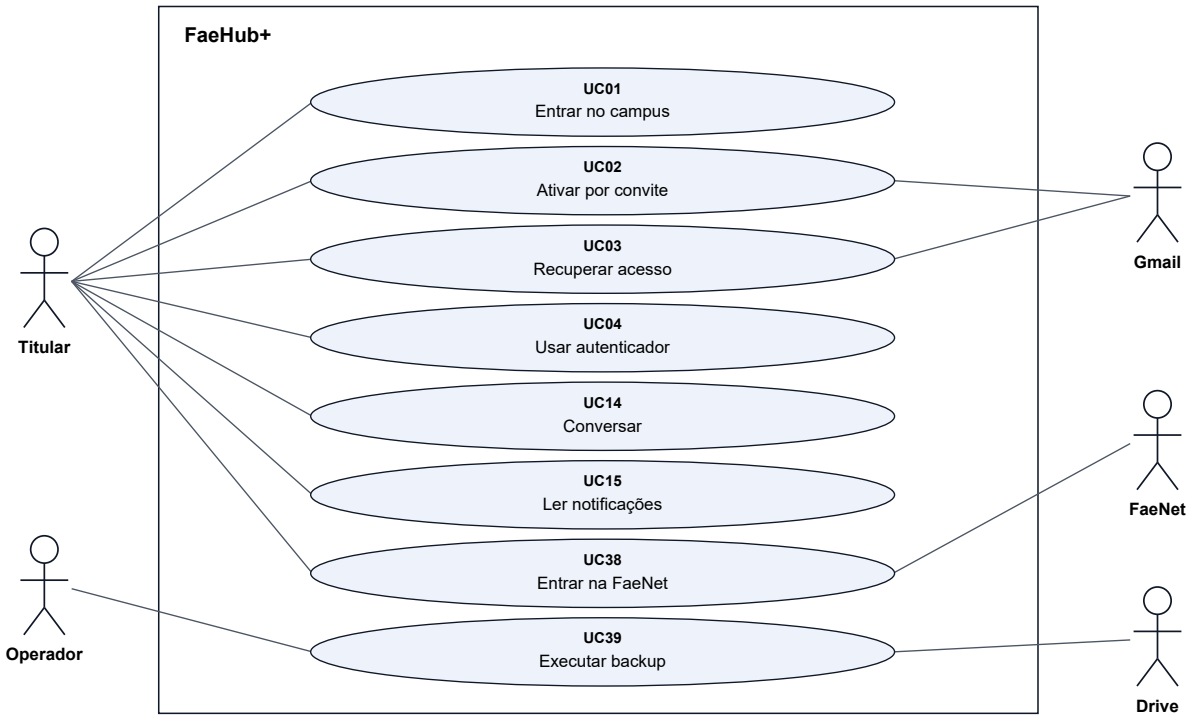
2.1.3. Regras de negócio

ID	Descrição
RN01	Turmas pertencem a um ano letivo e podem estar associadas a curso; disciplinas são ofertadas por turma.
RN02	Cada estudante pode possuir somente uma matrícula com situação ativa, preservando matrículas anteriores.
RN03	Os períodos pertencem ao ano letivo e possuem datas e situação próprias.
RN04	Avaliações do núcleo acadêmico pertencem a uma oferta de disciplina e a um período.
RN05	A consulta atual indica Aprovado a partir de média 6,0 e Recuperação abaixo disso; o rótulo não constitui decisão institucional de conclusão.
RN06	N1 e N2 devem ser finitas, entre 0 e 10; a média da consulta é (N1 + N2) / 2.
RN07	A chamada associa cada matrícula a uma aula identificada por disciplina e horário.
RN08	Somente chamadas registradas compõem a frequência. Ausência de dados não comprova presença ou falta.
RN09	O acesso é validado pelo servidor segundo papel e vínculos da conta.
RN10	Professor só atua nas turmas e disciplinas vinculadas; selecionar um perfil visual não concede permissões.
RN11	Áreas protegidas exigem autenticação e segunda confirmação quando MFA estiver ativo.
RN12	Conta inativa ou sessão revogada não deve manter acesso às áreas protegidas.
RN13	Chamadas exigem vínculo docente vigente, matrícula válida na data e período aberto; aulas futuras são rejeitadas.
RN14	Avisos e notificações respeitam destinatários, turmas e permissões do fluxo correspondente.
RN15	Solicitações de documentos recebem protocolo e só podem ser consultadas ou atendidas por usuários autorizados.
RN16	Vagas respeitam a turma, o prazo e o estado de arquivamento definidos na publicação.
RN17	Ações relevantes são auditadas; alterações concorrentes de chamada exigem revisão atualizada.
RN18	O funcionamento depende de PostgreSQL configurado. Backups de dados são separados do histórico do código.

3. CASOS DE USO

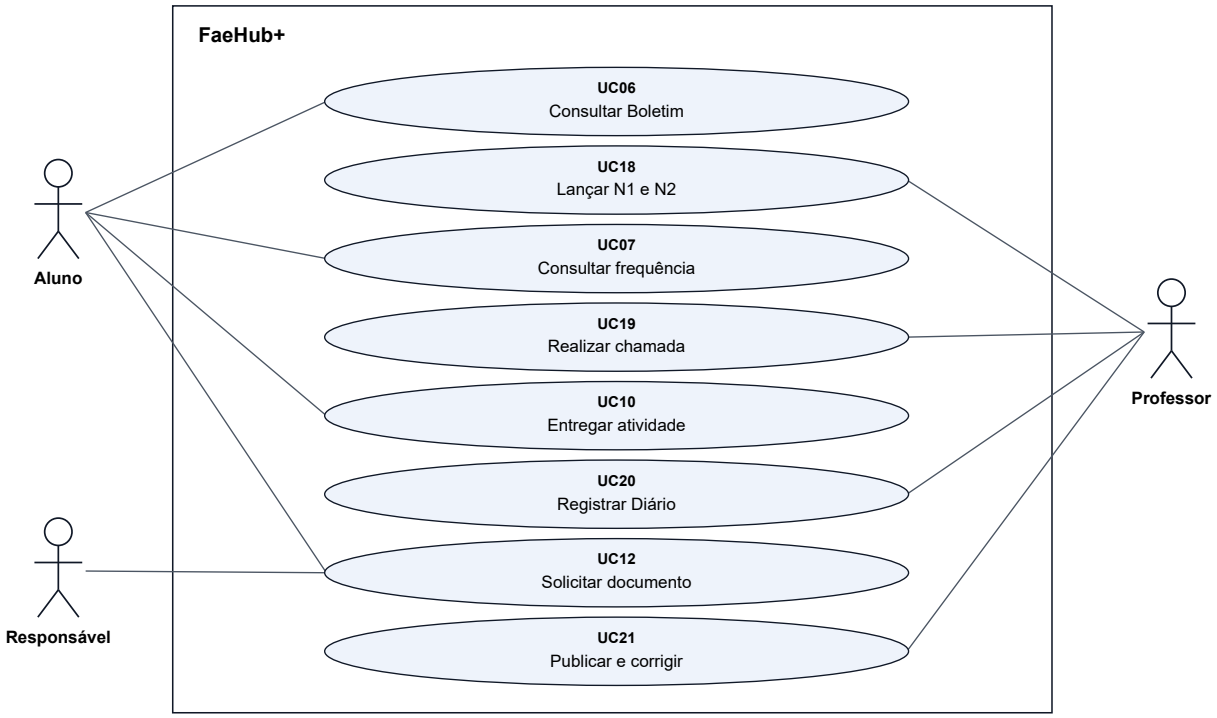
3.1. Diagrama de casos de uso

Casos de uso de acesso e serviços



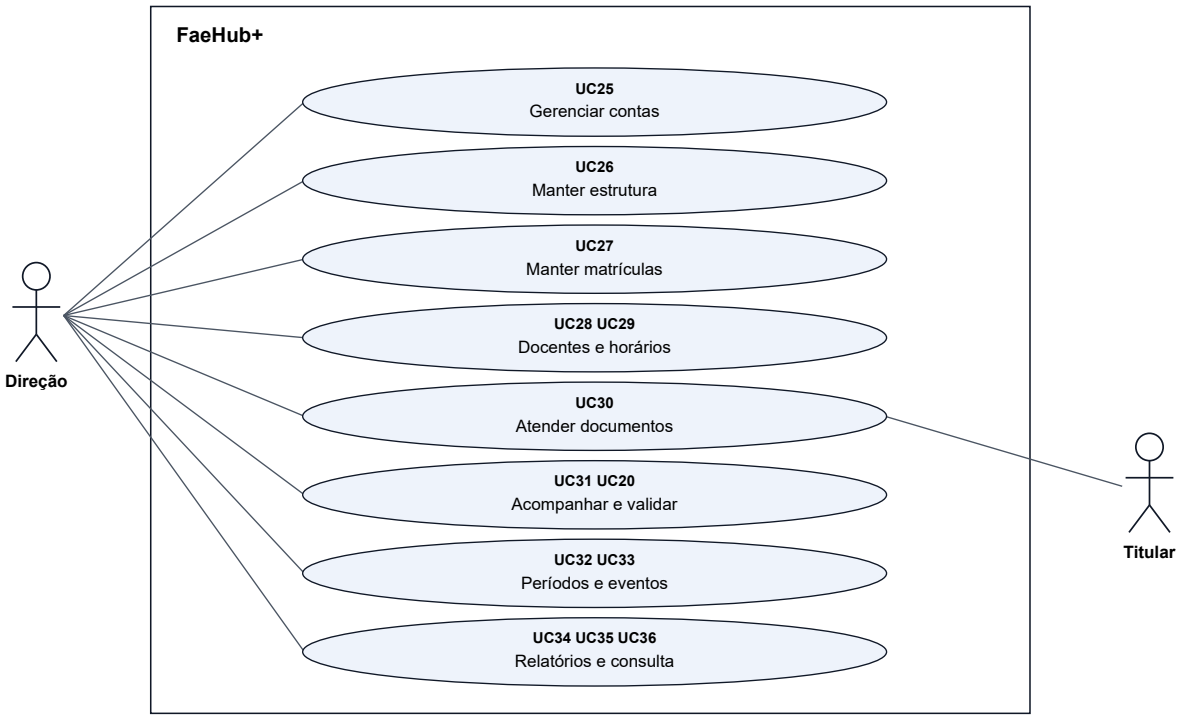
Titular representa os perfis com acesso permitido. Gmail participa de convite e recuperação; FaeNet é o aplicativo de destino; Drive é destino opcional da cópia criptografada. Backup é uma ferramenta operacional externa ao menu. As associações indicam participação, não sequência de execução.

Casos de uso acadêmicos



Aluno consulta seus registros e entrega atividades. Professor registra a aula, a presença, as notas e a correção. Responsável participa da solicitação para matrículas vinculadas e consulta resultados pelo seu portal, não pela edição de notas. Autenticação e vínculos válidos são pré-condições dos casos protegidos.

Casos de uso da gestão escolar



A direção mantém cadastros e atende os fluxos institucionais. O titular acompanha o documento solicitado. Algumas elipses agrupam casos relacionados para manter a visão legível; os identificadores individuais constam do catálogo. Não há uma etapa de aprovação da direção na recuperação por e-mail.

3.2. Especificação dos casos de uso

A conta deve estar ativa e autorizada para as operações protegidas. As exceções de autenticação e cadastro estão indicadas nos casos correspondentes.

Caso e ator	Fluxo e resultado esperado
UC01 — Entrar e sair do campus Todos os perfis	1. Abra o endereço público do FaeHub+ e escolha Aluno, Professor, Diretor ou Responsável. 2. Informe seu e-mail ou usuário e sua senha. Clique em Validar acesso. 3. Se a conta tiver autenticador ativo, conclua a segunda confirmação com o método oferecido. 4. Na confirmação de identidade, escolha Entrar no campus. Para encerrar, abra seu nome no cabeçalho e use Sair da conta. Conta bloqueada, perfil incompatível ou credenciais inválidas não liberam as áreas protegidas. Não compartilhe sua sessão.
UC02 — Ativar uma conta por convite Titular convidado	1. Peça à direção o cadastro com seu e-mail correto e os vínculos acadêmicos necessários. 2. Abra o link do convite na sua própria caixa de e-mail. A tela de Primeiro acesso sem convite não libera a continuação. 3. Confira a identidade apresentada, escolha uma senha entre 8 e 128 caracteres e repita a mesma senha. 4. Conclua a confirmação do e-mail e a ativação. Depois retorne ao login com as credenciais pessoais. Convites expirados, usados, revogados ou não enviados não devem ser reutilizados; solicite um novo envio à escola.
UC03 — Recuperar senha ou nome de usuário Titular com e-mail confirmado	1. No login, clique em Recuperar senha ou usuário e informe o e-mail confirmado. 2. Escolha Meu usuário para receber sua identificação por e-mail, ou Minha senha para receber um código de uso único. 3. Para redefinir, informe o mesmo e-mail e o código mais recente, válido por 30 minutos. Digite e confirme a nova senha. 4. Conclua a redefinição e entre novamente. Confira o spam se a mensagem não aparecer na caixa de entrada. Reenvio invalida o código anterior. Existem limites de tentativas; aguarde quando solicitado.

Caso e ator	Fluxo e resultado esperado
UC04 — Utilizar o autenticador da conta Todos os titulares ativados	1. Abra seu nome no cabeçalho e selecione Segurança da conta. Confirme que o servidor está configurado e que sua conta foi ativada por e-mail. 2. Confirme a senha atual, leia o QR no autenticador ou cadastre a chave manualmente e confirme um código de seis dígitos antes da expiração de dez minutos. 3. Guarde os dez códigos de recuperação em local privado. Eles são exibidos uma vez e cada um pode ser usado apenas uma vez, junto com a senha. 4. Para gerar novos códigos ou desativar o recurso, use a opção correspondente e confirme senha e um código novo. Nunca envie QR ou códigos por mensagens. A captura mostra o estado sem chave de proteção no ambiente local, não uma verificação da configuração de produção. Não é possível ativar enquanto essa pendência existir.
UC05 — Consultar o painel do aluno Aluno	1. Confira a identificação e a turma mostradas no início do painel. 2. Use Ver horário ou Abrir agenda para conferir disciplina, dia e início das próximas aulas. 3. Abra Ver boletim para analisar os lançamentos de notas. Confira a frequência na Agenda e seu histórico de chamadas. 4. Use Avisos da turma ou o cartão de trabalhos pendentes para abrir a lista correspondente. Contadores refletem os registros disponíveis. Um painel com zero e sem chamadas não comprova ausência ou desempenho insuficiente.
UC06 — Ler e imprimir o Boletim Aluno	1. Abra Boletim e confirme o estudante e a turma. Na tabela, compare N1, N2 e a média de cada disciplina. 2. Use Todas, Na média ou Em recuperação para filtrar. A referência exibida é 6,0 e a média da disciplina é (N1 + N2) / 2. 3. Abra Simular uma média para experimentar valores. A simulação não altera o boletim oficial nem grava notas. 4. Use Imprimir boletim para abrir a impressão do navegador e escolher uma impressora ou Salvar como PDF. O piloto visual da captura mostra barras de 0 a 10 com referência em 6,0. Sem lançamento, apresenta traço e mensagem de espera.
UC07 — Acompanhar Agenda e frequência Aluno	1. Abra Agenda, escolha o mês em Consultar mês e clique em Atualizar. 2. Selecione um dia para consultar o registro; confira as chamadas por disciplina e horário na lista do mês. 3. Leia os próximos eventos e observe o público, a data e o local publicados. 4. Se precisar de registros antigos, abra Histórico diário anterior. Use Consultar por aula para retornar à base atual. Ausência de chamada não é falta. Um mesmo dia pode reunir presenças e faltas em aulas distintas e aparecer com indicação mista.
UC08 — Interpretar a grade de horários Aluno	1. Abra Horário e confira a turma, o período abrangido e a próxima aula na grade. 2. Escolha Semana completa ou um dia específico para reduzir a visualização. 3. Leia cada bloco com início, término, disciplina e docente. Diferencie blocos de aula de Sem aula ou Tempo livre. 4. Use Preparar minhas entregas para ir a Exercícios. Havendo divergência na grade, informe a secretaria. Tempo livre significa ausência de aula publicada; não afirma que esse período seja um intervalo oficial.
UC09 — Buscar atividades e consultar devolutivas Aluno	1. Abra Exercícios e procure pelo título. Use Todas, Pendentes ou Entregues para restringir a lista. 2. Clique em Abrir atividade para ler o enunciado completo, os materiais e a data de entrega. 3. Depois do envio, retorne à atividade para conferir sua resposta, o anexo e uma eventual correção do professor. 4. Se o professor solicitar ajuste, revise a resposta com cuidado e confirme o novo envio pelo formulário. Prazo encerrado é um aviso. A interface permite entrega tardia, mas a aceitação depende do professor.

Caso e ator	Fluxo e resultado esperado
UC10 — Entregar uma atividade com anexo Aluno	1. Abra a atividade correta e leia suas instruções antes de preencher a resposta. 2. Digite o texto ou link solicitado. Para anexar, clique em Adicionar arquivo e escolha um arquivo dentro dos formatos e do tamanho permitidos. 3. Confira o nome do arquivo selecionado. Não envie documentos pessoais desnecessários ou links sem permissão de leitura para o professor. 4. Clique em Entregar atividade e confira a confirmação e a situação de entregue. Reabra para verificar o que ficou salvo. É necessária uma resposta ou um anexo. Escolher um arquivo sem concluir o envio não cria uma entrega.
UC11 — Ler e organizar os Comunicados Aluno	1. Abra Comunicados e leia o conteúdo de cada aviso ou atualização. 2. Use Marcar como lido ou Marcar como lida para registrar a leitura de um item. 3. Para registrar a leitura dos itens elegíveis de uma vez, use Marcar tudo como lido. 4. Nas atualizações, Abrir contexto leva ao documento, evento ou área relacionada. Quando disponível, o descarte de aviso pode ser desfeito. Marcar tudo como lido não apaga itens e não altera conversas de Mensagens nem comunicados descartados.
UC12 — Solicitar e acompanhar documentos Aluno e responsável autorizado	1. Abra Secretaria, confira a matrícula e selecione o tipo de documento. 2. Informe a finalidade e clique em Gerar protocolo. Anote o protocolo apresentado. 3. Acompanhe a situação e as observações da escola em Acompanhamento. 4. Quando a situação permitir, clique em Abrir. No documento emitido, use Imprimir / salvar PDF e confira a identificação do estudante. As situações incluem Solicitado, Em processamento, Pronto, Entregue e Indeferido. Pronto não significa que o documento já foi retirado.
UC13 — Consultar oportunidades de estágio Aluno	1. Abra Estágios e use a busca por oportunidade. 2. Filtre por Todas, Presencial, Híbrido ou Remoto. Confira o prazo de inscrição. 3. Abra o anúncio para ler empresa, descrição, requisitos, local, carga horária e instruções. 4. Inscreva-se pelo link ou canal indicado pelo anúncio, depois de verificar a confiabilidade do destino. Uma lista vazia pode indicar ausência de publicações ou filtros restritivos. Use Limpar filtros.
UC14 — Conversar com pessoas e grupos Todos os perfis conforme contatos autorizados	1. Abra Mensagens. Escolha uma conversa existente ou clique em Nova conversa. 2. Em uma nova conversa, selecione uma pessoa da lista permitida e escreva a mensagem. Para a turma, abra o grupo correspondente. 3. Use Adicionar foto ou arquivo ou Anexar para escolher uma imagem, PDF, TXT ou ZIP permitido. Confira a seleção e clique em Enviar. 4. Use a busca para reencontrar conversas. Abra a conversa para acompanhar as respostas e os indicadores de leitura disponibilizados. A interface informa otimização de imagens antes do envio e limite de até 5 MB para demais arquivos. O servidor também valida o arquivo.
UC15 — Gerenciar notificações da conta Professor direção e responsável	1. Abra Notificações no menu ou pelo sino do cabeçalho. 2. Use Todas, Não lidas e Lidas para consultar o histórico apresentado. 3. Abra o contexto de uma atualização quando houver um atalho para a área de origem. 4. Marque um item como lido ou utilize Marcar tudo como lido para os itens elegíveis da sua central. A tela informa até 100 notificações recentes. O botão em lote pode ficar desabilitado quando não há itens não lidos.
UC16 — Organizar o trabalho pelo painel docente Professor	1. Confira os totais de turmas e vínculos, sem confundir número de alunos com quantidade de contas de login. 2. Em Próxima aula, use Abrir chamada para iniciar o registro da turma correspondente. 3. Consulte Na sua mesa para abrir as entregas a corrigir e os lançamentos já salvos. 4. Use os atalhos Lançar notas, Registrar frequência e Criar atividade. Em Minhas turmas, confirme os componentes atribuídos. Os vínculos são administrados pela direção. Uma turma ausente do painel não deve ser acessada usando a conta de outro professor.

Caso e ator	Fluxo e resultado esperado
UC17 — Consultar as turmas vinculadas Professor	1. Abra Minhas turmas e selecione a turma disponível. 2. Confira a quantidade de alunos e os componentes curriculares vinculados. 3. Use Buscar aluno para localizar nome ou matrícula na lista. 4. Use Abrir chamada, Lançar notas ou o atalho Notas do estudante para abrir o contexto apropriado. A lista corresponde ao cadastro acadêmico. Vínculo de usuário e matrícula escolar são informações distintas.
UC18 — Lançar e corrigir notas Professor com vínculo válido	1. Selecione a turma e a disciplina e clique em Abrir boletins. 2. Localize o aluno pela busca e confira sua matrícula. 3. Preencha N1 e N2, confira a média prevista e clique em Salvar na mesma linha. 4. Após a confirmação, consulte o registro novamente. Repita a ação para outros estudantes; não presuma que salvar uma linha gravou todas. Não há divisão por bimestre no lançamento atual. A nota de uma correção de exercício não é copiada automaticamente para N1/N2.
UC19 — Realizar a chamada por aula Professor com vínculo vigente	1. Consulte turma e data. Selecione a disciplina e informe início e término da aula no horário da escola. 2. Para corrigir uma chamada existente, use Carregar chamada salva antes de editar. 3. Marque todos os estudantes. O atalho Marcar os alunos como presentes preenche a turma; depois marque as exceções como Ausente. A busca não reduz a obrigação de conferir a turma inteira. 4. Revise os contadores de presentes, ausentes e sem marcação. Clique em Salvar chamada desta aula e confirme os registros ao recarregar a mesma seleção. A aula não pode ser futura, sobreposta, fora da matrícula ou sem vínculo docente vigente. Turma e ano devem estar ativos e a data deve ter um único período aberto.
UC20 — Registrar e enviar o Diário Professor	1. Selecione turma, disciplina e data; informe os tempos de aula. 2. Descreva conteúdo e objetivos e complete os demais campos pedagógicos necessários. 3. Use Salvar rascunho se o registro ainda estiver em elaboração, ou Enviar para validação quando estiver concluído. 4. Acompanhe Meus registros. Caso seja devolvido com ajustes, leia a observação, corrija o registro permitido e envie novamente. As situações do fluxo incluem Rascunho, Em validação, Validado e Ajustes solicitados.
UC21 — Publicar atividades e corrigir entregas Professor	1. Use Criar atividade e selecione uma turma e disciplina vinculadas. Informe título, prazo e instruções; anexe material se necessário. 2. Clique em Publicar para os alunos. Depois, utilize a busca e os filtros Ativos, Arquivados ou Todos. 3. Abra Ver entregas e expanda o aluno. Leia resposta e arquivos, atribua a nota da atividade, escreva feedback e salve a correção. 4. Para retirar a atividade da lista ativa, use Arquivar. No filtro Arquivados, a ação de restauração permite reativá-la. Só o autor gerencia sua publicação. Alterar a resposta após a correção exige nova revisão; o formulário protege a referência da resposta corrigida.
UC22 — Publicar e arquivar avisos da turma Professor	1. Abra Avisos e clique em Novo comunicado. 2. Selecione a turma vinculada, escreva título e conteúdo e defina a prioridade. 3. Publique e confira a mensagem na lista de ativos. Use Editar para revisar uma publicação sua. 4. Arquive um comunicado que não deva continuar ativo. Use o filtro Arquivados e a restauração para disponibilizá-lo novamente. Alunos veem avisos ativos da própria turma. Prioridade não concede acesso a outras turmas.
UC23 — Publicar oportunidades profissionais Professor	1. Clique em Publicar oportunidade e selecione uma turma vinculada. 2. Preencha os dados da empresa e da vaga, o prazo e a modalidade Presencial, Híbrido ou Remoto. 3. Informe um link seguro ou instruções de inscrição e publique para os alunos. 4. Use busca e filtros de modalidade e visibilidade. Edite, arquive ou restaure suas publicações quando houver mudança de prazo ou disponibilidade. É obrigatório fornecer link seguro ou instruções utilizáveis. Confirme a legitimidade da empresa antes de divulgar.

Caso e ator	Fluxo e resultado esperado
UC24 — Acompanhar o observatório da direção Direção	1. Confira os totais e diferencie cadastro acadêmico, conta de acesso e vínculo docente. 2. Na Fila de atendimento, abra diários a validar, documentos em andamento ou acompanhamentos abertos. 3. Consulte calendário e panorama das turmas para localizar o contexto da atividade escolar. 4. Use Gerenciar comunidade para Usuários ou Ver relatórios para exportações e trilha de atividade. Os indicadores vêm de registros salvos. Sem registros não há uma tendência estatística comprovada.
UC25 — Gerenciar contas e convites Direção ativada e proprietário conforme autorização	1. Confirme que sua conta de direção está ativada e que o envio de e-mail está configurado. 2. Use Preparar convite para cadastrar a identificação e o perfil correto, com e-mail do titular e vínculos exigidos pelo formulário. 3. Em Gerenciar, confira os dados da conta antes de aplicar alterações, reenviar ou revogar um convite ou alterar a situação permitida. 4. Utilize busca e filtro de perfil para conferir a comunidade. A ação Encerrar contas de demonstração só deve ser usada após garantir o acesso pessoal da direção. Preparar um cadastro não garante envio de convite. Sem transporte de e-mail, a própria tela informa que a ativação não foi liberada.
UC26 — Organizar a estrutura acadêmica Direção	1. Abra Estrutura e consulte o Mapa para conhecer os cadastros já existentes. 2. Use Anos e períodos para criar o ciclo letivo, delimitar suas datas e cadastrar os períodos dentro desse ano. 3. Defina cursos e disciplinas, depois crie turmas e associe seus componentes. 4. Cadastre estudantes e matrículas e, por fim, os vínculos docentes e os horários. Use Consultar históricos para recuperar uma trajetória anterior. Alterar a situação não significa apagar registros. Ano, turma e matrícula encerrados continuam disponíveis para histórico.
UC26 — Cadastrar anos e períodos letivos Direção	1. Em Novo ano letivo, informe identificação e intervalo de datas e use Criar ano. 2. Abra Novo período e associe-o ao ano correto, com ordem e datas coerentes. 3. Confira na linha do tempo os períodos e a situação do ano antes de atualizar. 4. Para um fechamento ou reabertura operacional, use Períodos e registre uma justificativa auditável. Não abra períodos conflitantes para a mesma data de chamada. O registro de frequência exige um único período aberto.
UC26 — Manter cursos e disciplinas Direção	1. Consulte os códigos e nomes existentes para evitar duplicação. 2. Use Novo curso ou Nova disciplina, preencha os campos e confirme a criação. 3. Vincule a disciplina à turma em Turmas e grade; o simples cadastro no catálogo não faz essa associação. 4. Para deixar de utilizar um item, use a ação de desativação ou ativação correspondente, preservando os vínculos históricos. Desativar não equivale a excluir resultados passados. Não reutilize o código de uma disciplina para dar outro significado a registros antigos.
UC26 — Cadastrar turmas e compor grades Direção	1. Em Nova turma, selecione ano e curso e informe identificação, etapa, turno, sala e capacidade. 2. Crie a turma e confira o registro no inventário. 3. Na turma correta, escolha uma disciplina do catálogo e clique em Vincular. Repita para os demais componentes. 4. Salve alterações de situação apenas após verificar matrículas e operação do ano. Depois configure os vínculos docentes e horários. Grade curricular não é grade de horários: adicionar disciplina não define dia nem início da aula.
UC27 — Cadastrar estudantes e matrículas Direção	1. Pesquise por nome, matrícula, turma ou ano e percorra a paginação antes de criar um novo registro. 2. Cadastre o estudante quando ele ainda não existir no diretório. 3. Abra Nova matrícula e relacione o estudante à turma correta, com a data adequada. 4. Quando necessário, utilize Encerrar e informe o motivo ou situação de saída oferecida. Consulte o histórico para conferir a trajetória preservada. O modelo mantém uma matrícula ativa por estudante. Transferência ou conclusão não deve apagar a matrícula anterior.

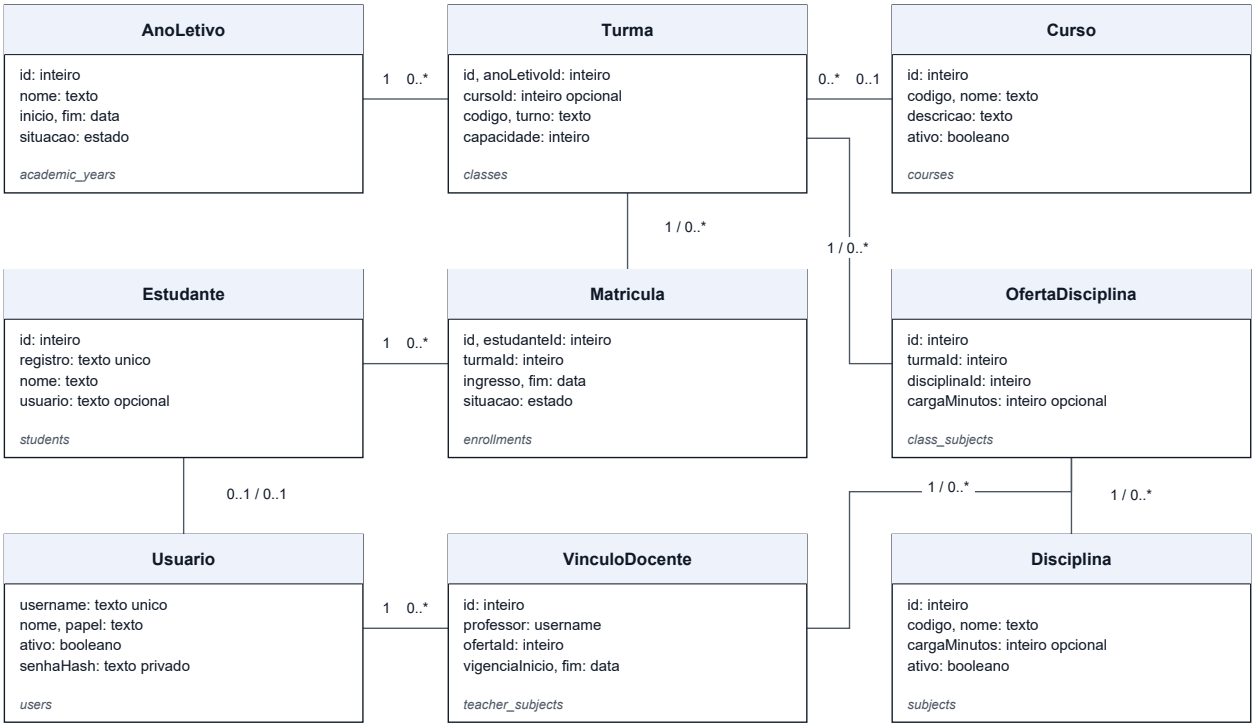
Caso e ator	Fluxo e resultado esperado
UC28 — Definir vínculos docentes Direção	1. Confira a conta ativa do professor e a oferta da disciplina na turma. 2. Selecione professor e combinação de turma e disciplina e informe a data de início. 3. Clique em Criar vínculo e confira o resultado no Mapa docente. 4. Quando houver mudança de responsabilidade, encerre o vínculo anterior e cadastre o novo respeitando as datas escolares. A chamada exige vínculo vigente na data da aula, não apenas um papel genérico de professor.
UC29 — Publicar e revisar horários semanais Direção	1. Selecione Turma e ano e use Consultar grade para visualizar os horários existentes. 2. Em Cadastrar horário, selecione a disciplina e informe dia, início, término e sala. 3. Clique em Salvar horário e confira a linha publicada. Use Editar para ajustar um registro existente. 4. Quando um horário deixar de vigorar, utilize Arquivar. Confira a visualização do aluno e os vínculos docentes depois da manutenção. O sistema verifica sobreposição de aulas da mesma turma, sala ou docente vinculado. Término deve ser posterior ao início.
UC30 — Atender e emitir documentos Direção	1. Localize o pedido na Central de protocolos e confira matrícula, tipo e finalidade. 2. Clique em Atualizar, escolha Em processamento, Pronto, Entregue ou Indeferido e escreva uma observação útil. 3. Use Salvar andamento e confira o estado exibido na lista. 4. Quando disponível, use Abrir para conferir o documento e Imprimir / salvar PDF. Marque Entregue somente após concluir a entrega pelo procedimento da escola. Solicitado é o estado inicial. Atualização gera histórico de situação e pode notificar o titular.
UC31 — Registrar acompanhamento pedagógico Direção	1. Consulte o Radar acadêmico e confira os registros do estudante antes de interpretar um alerta. 2. Em Novo acompanhamento, selecione o estudante e descreva categoria, ocorrência pedagógica e plano de ação. 3. Use Abrir acompanhamento e confira a entrada na lista de planos. 4. Registre as atualizações permitidas e acompanhe as situações Aberto, Em acompanhamento ou Fechado. Utilize Validar diários para acessar a fila pedagógica. Indicador de atenção não é uma decisão automática nem um diagnóstico. A equipe precisa contextualizar notas e frequência.
UC32 — Fechar e reabrir períodos Direção	1. Confira nome, ano e datas do período antes de qualquer alteração. 2. Use Fechar em um período aberto ou Reabrir em um período fechado. 3. Informe o motivo da alteração, entre 5 e 500 caracteres, no fluxo apresentado e confirme. 4. Confira a situação atual e a Trilha de auditoria, com ação, responsável, data e motivo. O fechamento protege registros alcançados pelas regras do período, inclusive chamadas e diário. A tela de notas atual não oferece uma nova divisão por bimestre.
UC33 — Publicar eventos no Calendário Direção publica e demais perfis consultam o público permitido	1. Consulte a Linha do tempo para evitar publicação repetida ou datas conflitantes. 2. Preencha título e intervalo de data e hora, selecione o tipo e defina o público correto. 3. Para público de turma, informe a identificação da turma. Complete local e descrição e use Publicar no calendário. 4. Confira a visibilidade no perfil correspondente. Se o evento deixar de ocorrer, utilize Cancelar evento no registro adequado. Professor e responsável consultam a linha do tempo autorizada; não recebem os controles de publicação da direção.
UC34 — Consultar relatórios e exportar registros Direção	1. Consulte as filas de diários, documentos, acompanhamentos e eventos futuros. 2. Para presença, informe as datas inicial e final e use Aplicar período. Limpar remove o recorte. 3. Escolha a exportação desejada: Diário de classe, Documentos, Acompanhamentos, Presenças do período ou Notas registradas. 4. Confira a base do percentual, os registros recentes e a Trilha de atividade. Abra o CSV em uma ferramenta compatível sem compartilhar o arquivo publicamente. A tabela de presença mostra até 30 registros e o CSV cobre o período completo selecionado. A trilha mostra as últimas 50 ações.

Caso e ator	Fluxo e resultado esperado
UC35 — Manter os dados institucionais Direção	1. Confira os valores atuais antes de editar. 2. Informe o nome da unidade, o ano de referência e o contato institucional corretos. 3. Clique em Salvar configurações e confira os valores após a confirmação. 4. Para usuários, vínculos, turmas ou períodos, abra as áreas correspondentes em vez de tentar controlar tudo pelo ano de referência. Mudar o ano de referência não apaga dados nem cria novo ano ou novo período de notas.
UC36 — Consultar a trajetória acadêmica Direção e titulares conforme matrícula autorizada	1. Na direção, abra Consultar históricos pelo Mapa da Estrutura ou Abrir consulta pela Secretaria. 2. Selecione o estudante autorizado e confira a identificação e o ano. 3. Leia a turma, a situação da matrícula e os resultados disponíveis para cada ciclo. 4. Para documento formal, solicite ou emita o tipo adequado pela Secretaria; a consulta visual não substitui a emissão. Ainda não existem resultados normalizados não significa que todos os campos N1/N2 do boletim sejam zero.
UC37 — Acompanhar estudantes vinculados Responsável	1. Entre pelo perfil Responsável com sua própria conta e confira a lista de estudantes vinculados. 2. Leia os resultados e a frequência do estudante correto, considerando se existem lançamentos e chamadas disponíveis. 3. Use Solicitar documento para abrir a Secretaria, Ver calendário para os eventos autorizados e Falar com a escola para Mensagens. 4. Se não houver matrícula vinculada, procure a escola para corrigir o cadastro. Não utilize as credenciais do estudante para contornar o vínculo. No painel fotografado, acompanhamentos aparecem como contagem; não é uma tela de edição de planos pedagógicos.
UC38 — Acessar a FaeNet pelo FaeHub Todos os perfis elegíveis	1. Abra FaeNet no menu e clique em Abrir a FaeNet ou no destino desejado. 2. Com a integração ativa, siga a entrada pelo FaeHub, autentique-se e conclua o autenticador se necessário. 3. Na primeira entrada da comunidade, personalize usuário, nome de exibição e, opcionalmente, foto, capa, bio e demais dados sociais. Vincular conta antiga exige a confirmação privada prevista pela FaeNet. 4. Depois, confira o destino na FaeNet. Sair dela encerra sua sessão social local, não necessariamente a sessão ainda válida do FaeHub. A captura local usa o estado sem configuração federada e informa acessos independentes. Não comprova nem desmente a ativação da integração em produção.
UC39 — Proteger os dados com backup Operador autorizado	Backup do banco é uma rotina operacional, não um botão disponível aos usuários escolares. O projeto contém ferramenta para criar cópia criptografada, verificar o arquivo e testar restauração em destino isolado. Não declarar backup automático operacional apenas porque a ferramenta existe. Exigir evidências da execução e de uma restauração isolada.

4. DIAGRAMA DE CLASSES

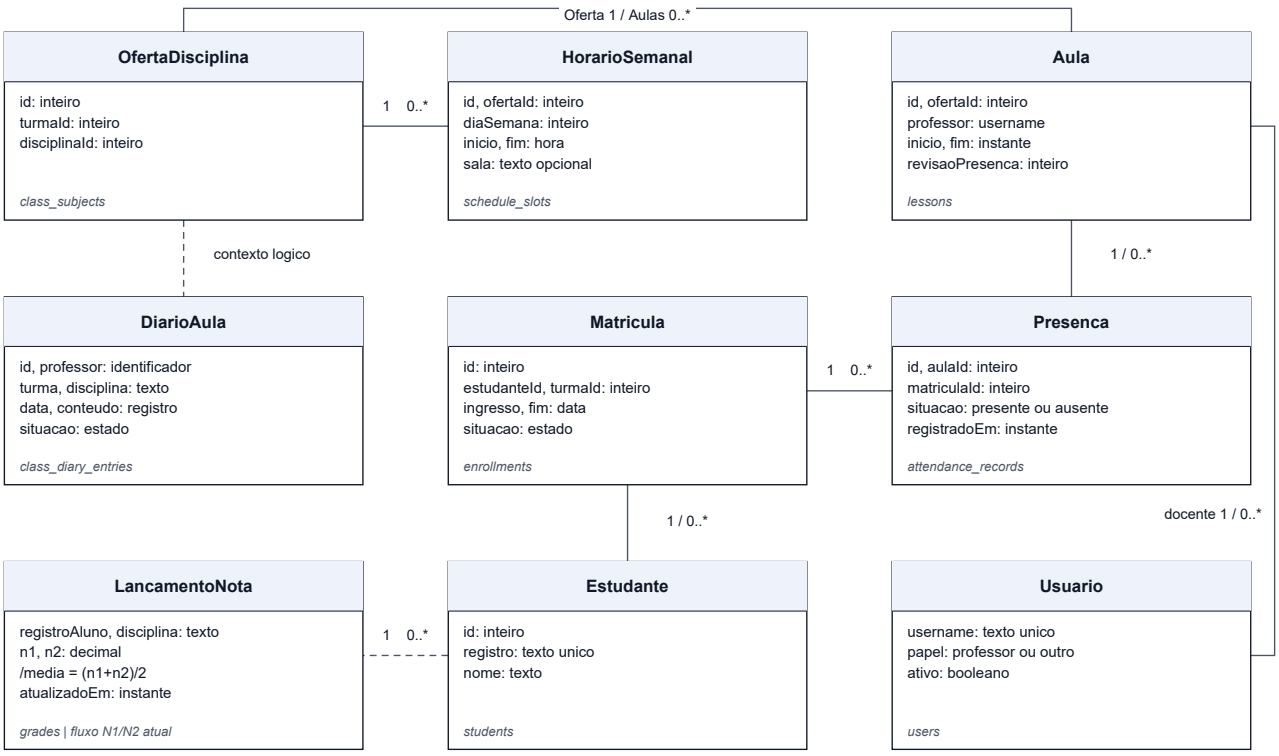
Os diagramas descrevem os conceitos do domínio e seus atributos. O código Flask implementa as operações em rotas e funções que consultam o banco.

Classes do núcleo acadêmico



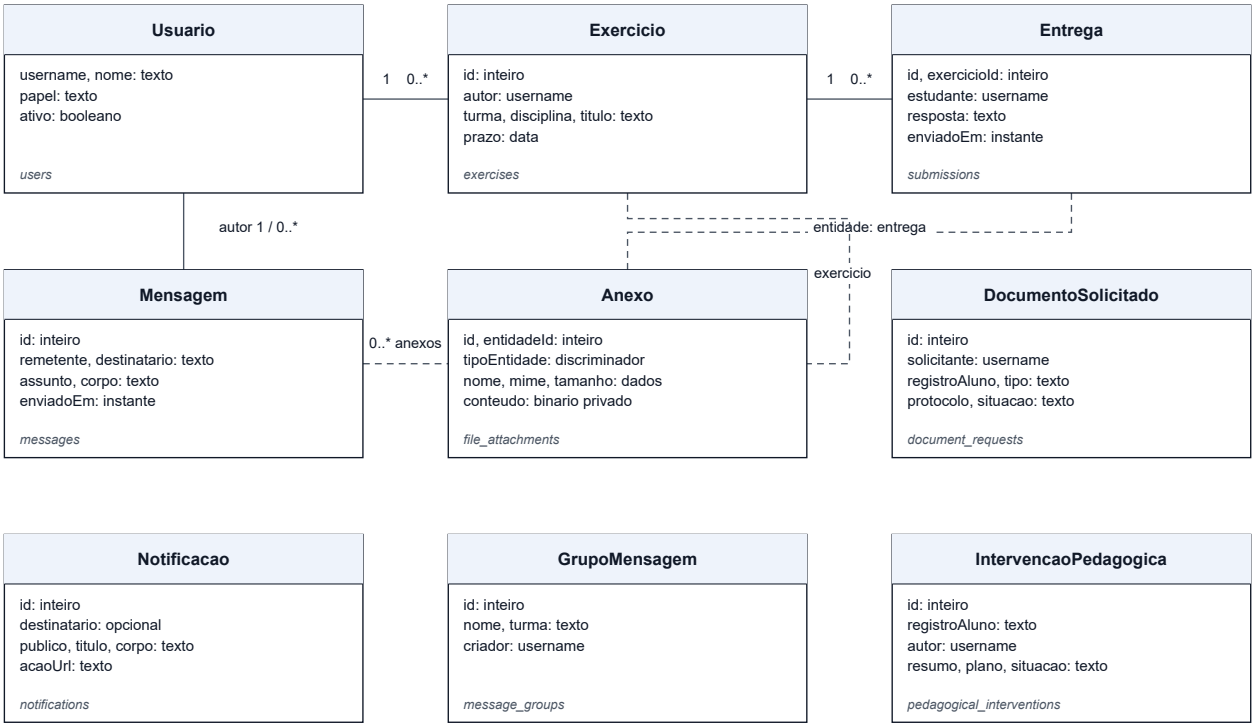
Modelo conceitual UML baseado em academic_core.py e database.py. A linha contínua representa associação; as multiplicidades aparecem entre as classes. As associações referem-se ao modelo de domínio, e não a herança entre papéis. Estudante e conta são distintos; uma matrícula ativa liga o estudante a uma turma.

Classes de aulas e resultados



Linha contínua: associação estrutural. Linha tracejada: relação lógica ou por código legado, sem afirmar uma chave estrangeira direta. A frequência atual relaciona Aula e Matricula. LancamentoNota representa o fluxo N1/N2 de grades; não deve ser confundido com assessments e assessment_scores, estruturas normalizadas presentes no projeto. /media é um valor derivado.

Classes de aprendizagem e serviços



Recorte conceitual dos principais registros de database.py e p1_operations.py. Anexo é polimórfico: tipoEntidade e entidadeId identificam mensagem, atividade ou entrega; as linhas tracejadas não são FKs diretas. Usuario também aparece nos atributos de autoria, solicitação e destinatário. Os vínculos auxiliares e o controle de leitura são detalhados na página seguinte.

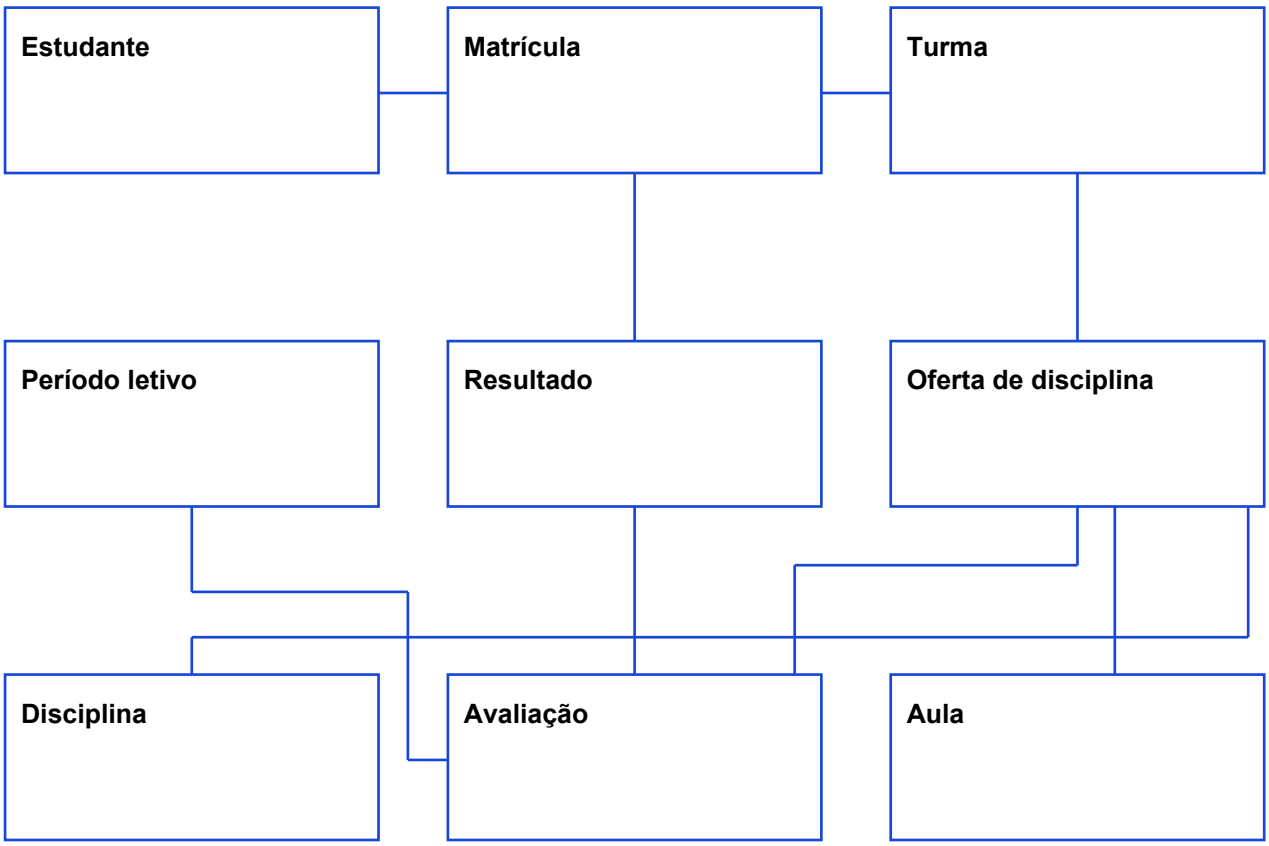
4.1. Especificação das classes

Classe ou conjunto	Responsabilidade
Conta, identidade, convite e autenticador	Representar o titular, seu acesso, a ativação e a segunda confirmação.
Ano letivo, período, curso e turma	Organizar calendário acadêmico e estrutura da formação.
Estudante, matrícula e responsável	Preservar o vínculo escolar e restringir o acompanhamento às matrículas autorizadas.
Disciplina, oferta e vínculo docente	Identificar a matéria ofertada em uma turma e a vigência da atuação do professor.
Avaliação, resultado, aula e frequência	Vincular notas e chamadas ao contexto acadêmico e à matrícula.
Atividade, entrega e correção	Organizar publicação, resposta e devolutiva do professor.
Mensagem, grupo e recibo	Registrar comunicação, participantes e leitura individual.
Documento, diário e intervenção	Acompanhar protocolos, registros de aula e ações pedagógicas.

5. PROJETO DO BANCO DE DADOS

5.1. Modelo Conceitual: Modelo de entidade e relacionamento

O recorte do núcleo acadêmico mostra como o estudante se vincula a uma turma, como a turma oferece disciplinas e como os resultados são associados a avaliações e matrículas.

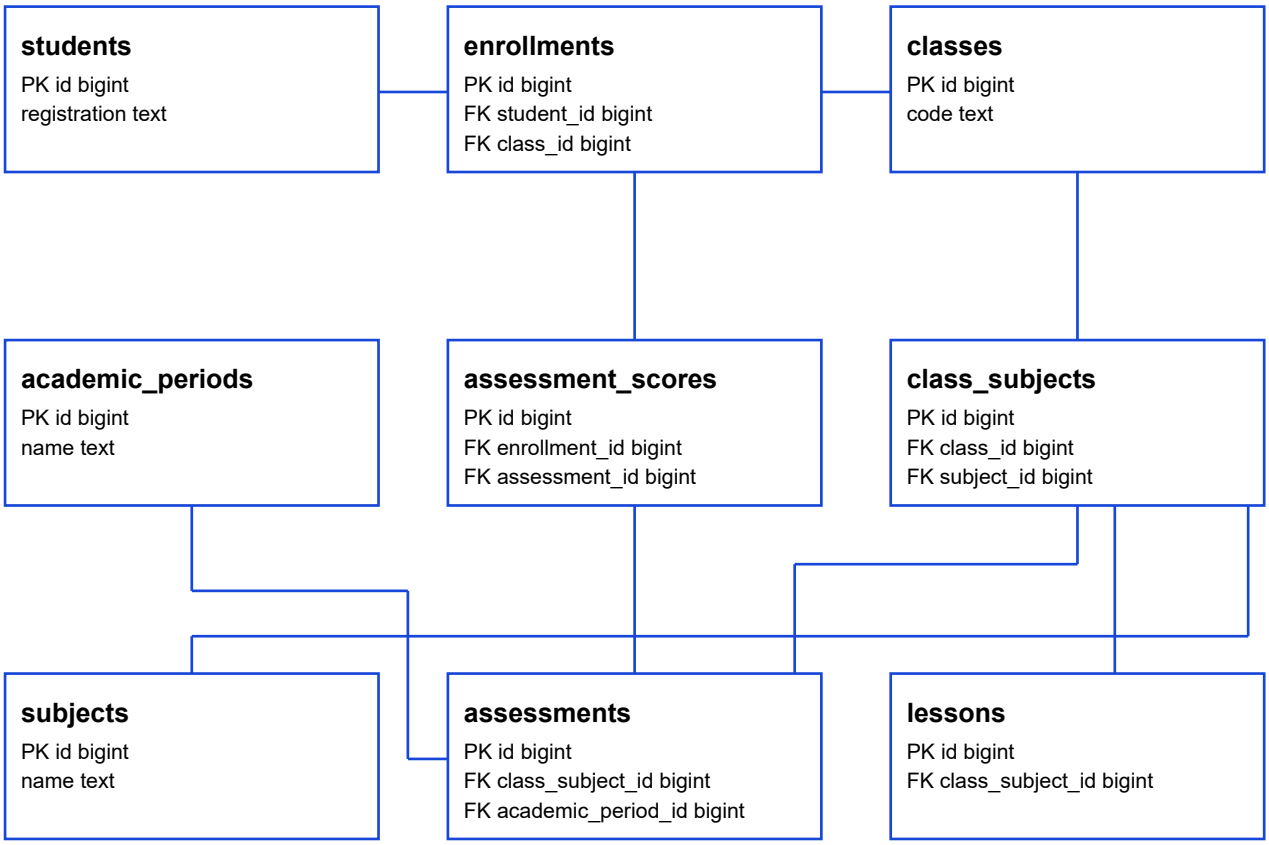


Entidade principal	Entidade dependente e cardinalidade
Estudante e turma	Cada matrícula pertence a um estudante e a uma turma. Cada um pode ter várias matrículas ao longo do tempo.
Turma e disciplina	Uma oferta vincula uma turma e uma disciplina. Cada turma ou disciplina pode participar de várias ofertas.
Oferta e período letivo	Uma avaliação pertence a uma oferta e a um período. A oferta também pode ter várias aulas.
Avaliação e matrícula	Cada resultado se vincula a uma avaliação e a uma matrícula.

5.2. Modelo físico

As migrações do PostgreSQL definem chaves, restrições, índices e permissões. O recorte abaixo usa os nomes e tipos das colunas do esquema.

5.2.1. Diagrama físico do banco de dados



PK indica chave primária e FK indica chave estrangeira. As linhas ligam as tabelas relacionadas. As referências completas, incluindo frequência por aula e vínculos docentes, aparecem no script.

5.2.2. Script SQL de criação do banco de dados

Aplique os blocos na ordem indicada em um projeto Supabase provisionado. As migrações usam os papéis e o esquema de autenticação fornecidos pelo serviço. A carga de demonstração foi omitida. O PDF contém também o script em texto no painel de anexos do leitor.

20260925151550_initial_school_schema.sql

```
-- FaeHub+ / Supabase PostgreSQL foundation.
-- The legacy-compatible tables keep the current Flask application operational
-- while the normalized academic model becomes the source of truth.

create table public.users (
  username text primary key,
  password_hash text not null,
  name text not null,
  role text not null check (role in ('diretor', 'professor', 'aluno')),
  student_id text unique,
  active boolean not null default true,
  auth_user_id uuid unique references auth.users(id) on delete set null,
  created_at timestamptz not null default now(),
  updated_at timestamptz not null default now()
);

create table public.grades (
  student_id text not null,
  discipline text not null,
  n1 numeric(5,2) not null check (n1 between 0 and 10),
  n2 numeric(5,2) not null check (n2 between 0 and 10),
  updated_at timestamptz not null default now(),
  primary key (student_id, discipline)
);

create table public.attendance (
  student_id text not null,
  class_date date not null,
  status text not null check (status in ('presente', 'ausente')),
  primary key (student_id, class_date)
);

create table public.activity_log (
  id bigint generated always as identity primary key,
  username text not null,
  action text not null,
  details text,
  created_at timestamptz not null default now()
);

create table public.institution_settings (
  key text primary key,
  value text not null
);

create table public.class_settings (
  class_name text primary key,
  capacity integer check (capacity is null or capacity > 0),
  shift text not null default 'Manhã'
    check (shift in ('Manhã', 'Tarde', 'Noite', 'Integral'))
);

create table public.auth_versions (
  username text primary key references public.users(username) on delete cascade,
  version integer not null default 0 check (version >= 0)
);

create table public.messages (
  id bigint generated always as identity primary key,
  sender text not null references public.users(username),
  recipient text not null references public.users(username),
  subject text not null,
  body text not null,
  is_read boolean not null default false,
  created_at timestamptz not null default now()
);

create table public.exercises (
```

```
  id bigint generated always as identity primary key,
  teacher text not null references public.users(username),
  title text not null,
  description text not null,
  discipline text not null,
  class_name text not null default '3110',
  due_date date not null,
  created_at timestamptz not null default now()
);

create table public.submissions (
  id bigint generated always as identity primary key,
  exercise_id bigint not null references public.exercises(id) on delete cascade,
  student text not null references public.users(username),
  answer text not null,
  submitted_at timestamptz not null default now(),
  unique (exercise_id, student)
);

create table public.teacher_assignments (
  teacher text not null references public.users(username) on delete cascade,
  class_name text not null,
  discipline text not null,
  primary key (teacher, class_name, discipline)
);

create table public.notices (
  id bigint generated always as identity primary key,
  teacher text not null references public.users(username),
  class_name text not null,
  title text not null,
  body text not null,
  priority text not null default 'normal'
    check (priority in ('normal', 'importante', 'urgente')),
  archived boolean not null default false,
  updated_at timestamptz not null default now()
);

create table public.exercise_states (
  exercise_id bigint primary key references public.exercises(id) on delete cascade,
  archived boolean not null default false
);

create table public.submission_reviews (
  submission_id bigint primary key references public.submissions(id) on delete cascade,
  feedback text not null,
  score numeric(5,2) not null check (score between 0 and 10),
  answer_snapshot text not null,
  reviewed_at timestamptz not null default now()
);

create table public.avatar_styles (
  username text primary key references public.users(username) on delete cascade,
  appearance text not null default 'masculine',
  hairstyle text not null default 'short',
  outfit text not null default 'hoodie',
  bottom text not null default 'trousers',
  shoes text not null default 'sneakers'
);

create table public.avatar_profiles (
  username text primary key references public.users(username) on delete cascade,
  skin text not null default '#d7a27d',
  hair text not null default '#172033',
  shirt text not null default '#367cf6',
  accessory text not null default 'none',
  updated_at timestamptz not null default now()
);

create table public.legacy_attendance_archive (
  legacy_student_id text not null,
  canonical_student_id text not null,
  class_date date not null,
  status text not null,
  migrated_at timestamptz not null default now(),
  primary key (legacy_student_id, class_date, status)
);
create table public.legacy_grade_archive (
```

```
  legacy_student_id text not null,
  canonical_student_id text not null,
  discipline text not null,
  n1 numeric(5,2) not null,
  n2 numeric(5,2) not null,
  source_updated_at timestamptz not null,
  migrated_at timestamptz not null default now(),
  primary key (legacy_student_id, discipline, source_updated_at, n1, n2)
);

create table public.seed_markers (
  name text primary key
);

-- Normalized academic model ------------------------------------------------

create table public.academic_years (
  id bigint generated always as identity primary key,
  name text not null unique,
  starts_on date not null,
  ends_on date not null,
  status text not null default 'planning'
    check (status in ('planning', 'active', 'closed', 'archived')),
  check (ends_on >= starts_on)
);

create table public.academic_periods (
  id bigint generated always as identity primary key,
  academic_year_id bigint not null references public.academic_years(id) on delete cascade,
  name text not null,
  period_number smallint not null check (period_number > 0),
  starts_on date not null,
  ends_on date not null,
  status text not null default 'planning'
    check (status in ('planning', 'open', 'closed')),
  unique (academic_year_id, period_number),
  check (ends_on >= starts_on)
);

create table public.courses (
  id bigint generated always as identity primary key,
  code text not null unique,
  name text not null,
  description text,
  active boolean not null default true
);

create table public.subjects (
  id bigint generated always as identity primary key,
  code text not null unique,
  name text not null,
  workload_minutes integer check (workload_minutes is null or workload_minutes > 0),
  active boolean not null default true
);

create table public.classes (
  id bigint generated always as identity primary key,
  academic_year_id bigint not null references public.academic_years(id),
  course_id bigint references public.courses(id),
  code text not null,
  grade_label text,
  shift text not null check (shift in ('Manhã', 'Tarde', 'Noite', 'Integral')),
  room text,
  capacity integer not null check (capacity > 0),
  status text not null default 'active'
    check (status in ('planning', 'active', 'closed', 'archived')),
  unique (academic_year_id, code)
);

create table public.class_subjects (
  id bigint generated always as identity primary key,
  class_id bigint not null references public.classes(id) on delete cascade,
  subject_id bigint not null references public.subjects(id),
  workload_minutes integer check (workload_minutes is null or workload_minutes > 0),
  unique (class_id, subject_id)
);

create table public.students (
```

```
  id bigint generated always as identity primary key,
  registration text not null unique,
  user_username text unique references public.users(username) on delete set null,
  full_name text not null,
  birth_date date,
  active boolean not null default true,
  created_at timestamptz not null default now(),
  updated_at timestamptz not null default now()
);

create table public.enrollments (
  id bigint generated always as identity primary key,
  student_id bigint not null references public.students(id),
  class_id bigint not null references public.classes(id),
  status text not null default 'active'
    check (status in ('active', 'transferred', 'withdrawn', 'completed', 'cancelled')),
  enrolled_on date not null default current_date,
  ended_on date,
  created_at timestamptz not null default now(),
  check (ended_on is null or ended_on >= enrolled_on)
);

create unique index enrollments_one_active_class_idx
  on public.enrollments (student_id, class_id)
  where status = 'active';

create table public.teacher_subjects (
  id bigint generated always as identity primary key,
  teacher_username text not null references public.users(username),
  class_subject_id bigint not null references public.class_subjects(id) on delete cascade,
  valid_from date not null default current_date,
  valid_until date,
  unique (teacher_username, class_subject_id, valid_from),
  check (valid_until is null or valid_until >= valid_from)
);

create table public.schedule_slots (
  id bigint generated always as identity primary key,
  class_subject_id bigint not null references public.class_subjects(id) on delete cascade,
  weekday smallint not null check (weekday between 1 and 7),
  starts_at time not null,
  ends_at time not null,
  room text,
  check (ends_at > starts_at),
  unique (class_subject_id, weekday, starts_at)
);

create table public.assessments (
  id bigint generated always as identity primary key,
  class_subject_id bigint not null references public.class_subjects(id) on delete cascade,
  academic_period_id bigint not null references public.academic_periods(id),
  title text not null,
  assessment_type text not null,
  max_score numeric(6,2) not null default 10 check (max_score > 0),
  weight numeric(7,3) not null default 1 check (weight > 0),
  due_on date,
  published_at timestamptz,
  created_at timestamptz not null default now()
);

create table public.assessment_scores (
  id bigint generated always as identity primary key,
  assessment_id bigint not null references public.assessments(id) on delete cascade,
  enrollment_id bigint not null references public.enrollments(id) on delete cascade,
  score numeric(6,2),
  status text not null default 'pending'
    check (status in ('pending', 'graded', 'absent', 'excused')),
  feedback text,
  published_at timestamptz,
  updated_at timestamptz not null default now(),
  unique (assessment_id, enrollment_id),
  check (score is null or score >= 0)
);

create table public.lessons (
  id bigint generated always as identity primary key,
  class_subject_id bigint not null references public.class_subjects(id),
  teacher_username text references public.users(username),
```

```
  starts_at timestamptz not null,
  ends_at timestamptz not null,
  topic text,
  status text not null default 'scheduled'
    check (status in ('scheduled', 'completed', 'cancelled')),
  check (ends_at > starts_at)
);

create table public.attendance_records (
  id bigint generated always as identity primary key,
  lesson_id bigint not null references public.lessons(id) on delete cascade,
  enrollment_id bigint not null references public.enrollments(id) on delete cascade,
  status text not null check (status in ('present', 'absent', 'late', 'excused', 'dismissed')),
  note text,
  recorded_at timestamptz not null default now(),
  unique (lesson_id, enrollment_id)
);

-- Foreign keys are not indexed automatically by PostgreSQL.
create index messages_sender_idx on public.messages (sender, created_at desc);
create index messages_recipient_idx on public.messages (recipient, created_at desc);
create index exercises_teacher_idx on public.exercises (teacher);
create index exercises_class_due_idx on public.exercises (class_name, due_date);
create index submissions_student_idx on public.submissions (student);
create index teacher_assignments_class_idx on public.teacher_assignments (class_name, discipline);
create index notices_class_updated_idx on public.notices (class_name, updated_at desc);
create index academic_periods_year_idx on public.academic_periods (academic_year_id);
create index classes_year_idx on public.classes (academic_year_id);
create index classes_course_idx on public.classes (course_id);
create index class_subjects_class_idx on public.class_subjects (class_id);
create index class_subjects_subject_idx on public.class_subjects (subject_id);
create index enrollments_student_idx on public.enrollments (student_id);
create index enrollments_class_idx on public.enrollments (class_id);
create index teacher_subjects_teacher_idx on public.teacher_subjects (teacher_username);
create index teacher_subjects_class_subject_idx on public.teacher_subjects (class_subject_id);
create index schedule_slots_class_subject_idx on public.schedule_slots (class_subject_id);
create index assessments_class_subject_idx on public.assessments (class_subject_id);
create index assessments_period_idx on public.assessments (academic_period_id);
create index assessment_scores_enrollment_idx on public.assessment_scores (enrollment_id);
create index lessons_class_subject_starts_idx on public.lessons (class_subject_id, starts_at);
create index lessons_teacher_idx on public.lessons (teacher_username);
create index attendance_records_enrollment_idx on public.attendance_records (enrollment_id);

-- Phase one is server-side only. Data API access remains closed until Supabase
-- Auth is connected and ownership-aware policies can be introduced safely.
do $$
declare
  table_name text;
begin
  foreach table_name in array array[
    'users','grades','attendance','activity_log','institution_settings','class_settings',
    'auth_versions','messages','exercises','submissions','teacher_assignments','notices',
    'exercise_states','submission_reviews','avatar_styles','avatar_profiles',
    'legacy_attendance_archive','legacy_grade_archive','seed_markers','academic_years',
    'academic_periods','courses','subjects','classes','class_subjects','students',
    'enrollments','teacher_subjects','schedule_slots','assessments','assessment_scores',
    'lessons','attendance_records'
  ]
  loop
    execute format('alter table public.%I enable row level security', table_name);
    execute format('revoke all on table public.%I from anon, authenticated', table_name);
  end loop;
end $$;

revoke all on all sequences in schema public from anon, authenticated;
alter default privileges in schema public revoke all on tables from anon, authenticated;
alter default privileges in schema public revoke all on sequences from anon, authenticated;
```

20260925151938_add_notices_teacher_index.sql

```
-- Índice recomendado pelo advisor do Supabase para a chave estrangeira.
create index notices_teacher_idx on public.notices (teacher);
```

20260925155251_add_internships_module.sql

```
create table public.internships (
  id bigint generated by default as identity primary key,
  teacher text not null references public.users(username),
  company text not null check (char_length(company) between 1 and 120),
  title text not null check (char_length(title) between 1 and 160),
  description text not null check (char_length(description) between 1 and 10000),
  location text not null check (char_length(location) between 1 and 160),
  modality text not null check (modality in ('presencial', 'hibrido', 'remoto')),
  workload text not null default '' check (char_length(workload) <= 80),
  requirements text not null default '' check (char_length(requirements) <= 5000),
  application_url text check (
    application_url is null
    or (char_length(application_url) <= 2048 and application_url ~ '^https?://')
  ),
  application_instructions text not null default '' check (char_length(application_instructions) <=
3000),
  deadline date not null,
  class_name text not null default '3110',
  archived boolean not null default false,
  created_at timestamptz not null default clock_timestamp(),
  updated_at timestamptz not null default clock_timestamp(),
  check (application_url is not null or char_length(application_instructions) > 0)
);

create index internships_teacher_idx
  on public.internships (teacher, archived, updated_at desc);

create index internships_class_deadline_idx
  on public.internships (class_name, archived, deadline);

alter table public.internships enable row level security;
revoke all on table public.internships from anon, authenticated;
revoke all on sequence public.internships_id_seq from anon, authenticated;
```

20260925191206_p1_school_operations.sql

```
-- FaeHub+ P1: operação escolar completa.
-- O Flask continua como única camada de acesso; a Data API permanece fechada.

create table public.account_roles (
  username text primary key references public.users(username) on delete cascade,
  role text not null check (role in ('responsavel','secretaria','coordenacao'))
);

create table public.guardian_links (
  id bigint generated by default as identity primary key,
  guardian_username text not null references public.users(username) on delete cascade,
  student_id text not null,
  relationship text not null,
  active boolean not null default true,
  unique (guardian_username,student_id)
);

create table public.period_closure_log (
  id bigint generated by default as identity primary key,
  period_id bigint not null references public.academic_periods(id),
  action text not null check (action in ('close','reopen')),
  actor text not null references public.users(username),
  reason text not null check (char_length(reason) between 5 and 500),
  created_at timestamptz not null default now()
);

create table public.class_diary_entries (
  id bigint generated by default as identity primary key,
  teacher text not null references public.users(username),
  class_name text not null,
  discipline text not null,
  lesson_date date not null,
  content text not null check (char_length(content) between 1 and 10000),
  objectives text not null check (char_length(objectives) between 1 and 5000),
  methodology text not null default '' check (char_length(methodology) <= 3000),
  resources text not null default '' check (char_length(resources) <= 2000),
  homework text not null default '' check (char_length(homework) <= 3000),
  observations text not null default '' check (char_length(observations) <= 5000),
  class_hours smallint not null default 2 check (class_hours between 1 and 12),
  status text not null default 'draft' check (status in ('draft','submitted','validated','returned')),
```

```
  coordinator_note text not null default '' check (char_length(coordinator_note) <= 3000),
  validated_by text references public.users(username),
  created_at timestamptz not null default now(),
  updated_at timestamptz not null default now()
);

create table public.document_requests (
  id bigint generated by default as identity primary key,
  requester text not null references public.users(username),
  student_id text not null,
  document_type text not null check (document_type in ('matricula','frequencia','historico','ficha')),
  purpose text not null default '' check (char_length(purpose) <= 500),
  status text not null default 'requested' check (status in
('requested','processing','ready','delivered','rejected')),
  protocol text not null unique,
  verification_code text not null,
  note text not null default '' check (char_length(note) <= 1000),
  requested_at timestamptz not null default now(),
  updated_at timestamptz not null default now()
);

create table public.pedagogical_interventions (
  id bigint generated by default as identity primary key,
  student_id text not null,
  opened_by text not null references public.users(username),
  category text not null check (char_length(category) between 1 and 80),
  summary text not null check (char_length(summary) between 5 and 1000),
  plan text not null check (char_length(plan) between 5 and 3000),
  status text not null default 'open' check (status in ('open','monitoring','closed')),
  created_at timestamptz not null default now(),
  updated_at timestamptz not null default now()
);

create table public.notifications (
  id bigint generated by default as identity primary key,
  recipient text references public.users(username),
  role_target text,
  title text not null check (char_length(title) between 1 and 160),
  body text not null check (char_length(body) between 1 and 2000),
  category text not null default 'sistema',
  action_url text not null default '' check (char_length(action_url) <= 500),
  created_by text references public.users(username),
  created_at timestamptz not null default now(),
  expires_at timestamptz,
  check (recipient is not null or role_target is not null)
);

create table public.notification_reads (
  notification_id bigint not null references public.notifications(id) on delete cascade,
  username text not null references public.users(username) on delete cascade,
  read_at timestamptz not null default now(),
  primary key (notification_id,username)
);

create table public.calendar_events (
  id bigint generated by default as identity primary key,
  title text not null check (char_length(title) between 1 and 160),
  description text not null default '' check (char_length(description) <= 5000),
  starts_at timestamptz not null,
  ends_at timestamptz not null,
  event_type text not null check (event_type in
('evento','feriado','recesso','avaliacao','reuniao','conselho')),
  audience text not null default 'todos' check (audience in
('todos','aluno','professor','admin','responsavel','turma')),
  class_name text not null default '',
  location text not null default '' check (char_length(location) <= 160),
  created_by text not null references public.users(username),
  status text not null default 'published' check (status in ('published','cancelled')),
  created_at timestamptz not null default now(),
  check (ends_at >= starts_at),
  check (audience <> 'turma' or char_length(class_name) > 0)
);

create table public.password_recovery_requests (
  id bigint generated by default as identity primary key,
  username text not null references public.users(username),
  token_hash text,
  status text not null default 'requested' check (status in ('requested','issued','used','expired')),
```

```
  requested_at timestamptz not null default now(),
  expires_at timestamptz,
  issued_by text references public.users(username),
  used_at timestamptz
);

create table public.file_attachments (
  id bigint generated by default as identity primary key,
  owner text not null references public.users(username),
  entity_type text not null check (entity_type in ('message','exercise','submission')),
  entity_id bigint not null,
  filename text not null,
  mime_type text not null,
  size_bytes integer not null check (size_bytes between 1 and 5242880),
  content bytea not null,
  created_at timestamptz not null default now()
);

create index guardian_links_student_idx on public.guardian_links (student_id,active);
create index period_closure_log_period_idx on public.period_closure_log (period_id,created_at desc);
create index period_closure_log_actor_idx on public.period_closure_log (actor);
create index diary_teacher_date_idx on public.class_diary_entries (teacher,lesson_date desc);
create index diary_status_idx on public.class_diary_entries (status,lesson_date desc);
create index diary_validated_by_idx on public.class_diary_entries (validated_by);
create index document_requests_requester_idx on public.document_requests (requester,requested_at desc);
create index document_requests_student_idx on public.document_requests (student_id,status,requested_at
desc);
create index interventions_opened_by_idx on public.pedagogical_interventions (opened_by);
create index interventions_student_idx on public.pedagogical_interventions (student_id,status,updated_at
 desc);
create index notifications_recipient_idx on public.notifications (recipient,created_at desc);
create index notifications_role_idx on public.notifications (role_target,created_at desc);
create index notifications_created_by_idx on public.notifications (created_by);
create index notification_reads_username_idx on public.notification_reads (username,read_at desc);
create index calendar_events_start_idx on public.calendar_events (starts_at,status);
create index calendar_events_created_by_idx on public.calendar_events (created_by);
create index password_recovery_status_idx on public.password_recovery_requests (status,requested_at
desc);
create index password_recovery_username_idx on public.password_recovery_requests (username,requested_at
desc);
create index password_recovery_issued_by_idx on public.password_recovery_requests (issued_by);
create index attachments_entity_idx on public.file_attachments (entity_type,entity_id);
create index attachments_owner_idx on public.file_attachments (owner,created_at desc);

do $$
declare table_name text;
begin
  foreach table_name in array array[
    'account_roles','guardian_links','period_closure_log','class_diary_entries',
    'document_requests','pedagogical_interventions','notifications','notification_reads',
    'calendar_events','password_recovery_requests','file_attachments'
  ] loop
    execute format('alter table public.%I enable row level security', table_name);
    execute format('revoke all on table public.%I from anon, authenticated', table_name);
  end loop;
end $$;

revoke all on all sequences in schema public from anon, authenticated;
```

20260926133712_complete_p0_academic_core.sql

```
-- P0 integrity: one current class per student and idempotent consolidated grades.
drop index if exists public.enrollments_one_active_class_idx;

create unique index if not exists enrollments_one_active_student_idx
  on public.enrollments (student_id)
  where status = 'active';

create unique index if not exists assessments_class_period_title_idx
  on public.assessments (class_subject_id, academic_period_id, title);

create index if not exists academic_years_status_starts_idx
  on public.academic_years (status, starts_on desc);

create index if not exists enrollments_class_status_idx
  on public.enrollments (class_id, status);
```

20260927123000_remove_avatar_system.sql

```
drop table if exists public.avatar_styles;
drop table if exists public.avatar_profiles;
```

20260928201235_communication_preferences.sql

```
-- Preferências individuais de leitura e descarte reversível de comunicados.
-- O Flask acessa a tabela pelo servidor; a Data API pública permanece fechada.

create table public.communication_states (
  username text not null references public.users(username) on delete cascade,
  kind text not null check (kind in ('notice','notification')),
  item_key text not null,
  is_read boolean not null default false,
  dismissed_at timestamptz,
  updated_at timestamptz not null default now(),
  primary key (username,kind,item_key)
);

create index communication_states_user_idx
  on public.communication_states (username,kind,dismissed_at);

alter table public.communication_states enable row level security;
revoke all on table public.communication_states from anon, authenticated;
```

20260929145350_secure_rich_messages.sql

```
-- Mensagens apagadas permanecem auditáveis e podem ser restauradas pelo autor.
alter table public.messages add column if not exists deleted_at timestamptz;
alter table public.messages add column if not exists deleted_by text references public.users(username);

create index if not exists messages_participants_created_idx
  on public.messages (sender, recipient, created_at desc);
create index if not exists messages_deleted_by_idx
  on public.messages (deleted_by) where deleted_by is not null;
```

20260929170000_complete_messaging.sql

```
-- Complete FaeHub+ messaging: groups, replies, edits, reactions and receipts.
ALTER TABLE public.messages ADD COLUMN IF NOT EXISTS group_id bigint;
ALTER TABLE public.messages ADD COLUMN IF NOT EXISTS reply_to_id bigint REFERENCES public.messages(id)
ON DELETE SET NULL;
ALTER TABLE public.messages ADD COLUMN IF NOT EXISTS edited_at timestamptz;
CREATE TABLE IF NOT EXISTS public.message_groups (
  id bigint GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  name text NOT NULL CHECK (char_length(name) BETWEEN 1 AND 80),
  class_name text,
  created_by text NOT NULL REFERENCES public.users(username),
  created_at timestamptz NOT NULL DEFAULT now()
);

DO $$ BEGIN
  IF NOT EXISTS (
    SELECT 1 FROM pg_constraint WHERE conname = 'messages_group_id_fkey'
  ) THEN
    ALTER TABLE public.messages ADD CONSTRAINT messages_group_id_fkey
      FOREIGN KEY (group_id) REFERENCES public.message_groups(id) ON DELETE CASCADE;
  END IF;
END $$;

CREATE TABLE IF NOT EXISTS public.message_group_members (
  group_id bigint NOT NULL REFERENCES public.message_groups(id) ON DELETE CASCADE,
  username text NOT NULL REFERENCES public.users(username) ON DELETE CASCADE,
  joined_at timestamptz NOT NULL DEFAULT now(),
  PRIMARY KEY (group_id, username)
);

CREATE TABLE IF NOT EXISTS public.message_receipts (
  message_id bigint NOT NULL REFERENCES public.messages(id) ON DELETE CASCADE,
  username text NOT NULL REFERENCES public.users(username) ON DELETE CASCADE,
  delivered_at timestamptz,
  read_at timestamptz,
  PRIMARY KEY (message_id, username),
  CHECK (read_at IS NULL OR delivered_at IS NOT NULL)
);

CREATE TABLE IF NOT EXISTS public.message_reactions (
```

```
  message_id bigint NOT NULL REFERENCES public.messages(id) ON DELETE CASCADE,
  username text NOT NULL REFERENCES public.users(username) ON DELETE CASCADE,
  emoji text NOT NULL CHECK (emoji IN ('■','❤■','■','■','■','■')),
  created_at timestamptz NOT NULL DEFAULT now(),
  PRIMARY KEY (message_id, username, emoji)
);

CREATE INDEX IF NOT EXISTS messages_direct_thread_idx
  ON public.messages(sender, recipient, id DESC) WHERE group_id IS NULL;
CREATE INDEX IF NOT EXISTS messages_group_thread_idx
  ON public.messages(group_id, id DESC) WHERE group_id IS NOT NULL;
CREATE INDEX IF NOT EXISTS message_group_members_user_idx
  ON public.message_group_members(username, group_id);
CREATE INDEX IF NOT EXISTS message_receipts_user_idx
  ON public.message_receipts(username, read_at, message_id);
CREATE INDEX IF NOT EXISTS message_reactions_message_idx
  ON public.message_reactions(message_id);

-- Carga de demonstração omitida.

INSERT INTO public.message_receipts(message_id, username, delivered_at, read_at)
SELECT id, recipient,
       CASE WHEN is_read THEN created_at END,
       CASE WHEN is_read THEN created_at END
FROM public.messages
WHERE group_id IS NULL
ON CONFLICT DO NOTHING;

COMMENT ON TABLE public.message_receipts IS
  'Per-user delivery and read state for direct and group messages.';
```

20260929190500_secure_messaging_tables.sql

```
-- The Flask server is the sole data authority; browser roles cannot query messaging tables.
ALTER TABLE public.message_groups ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.message_group_members ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.message_receipts ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.message_reactions ENABLE ROW LEVEL SECURITY;
CREATE INDEX IF NOT EXISTS message_groups_created_by_idx
  ON public.message_groups(created_by);
CREATE INDEX IF NOT EXISTS message_reactions_username_idx
  ON public.message_reactions(username);
CREATE INDEX IF NOT EXISTS messages_reply_to_idx
  ON public.messages(reply_to_id) WHERE reply_to_id IS NOT NULL;
```

20261008153305_profile_photos.sql

```
-- Photos remain private to the Flask account session. No public Data API access.
CREATE TABLE public.profile_photos (
    username TEXT PRIMARY KEY REFERENCES public.users(username) ON DELETE CASCADE,
    content BYTEA NOT NULL CHECK(octet_length(content) BETWEEN 1 AND 196608),
    revision TEXT NOT NULL,
    updated_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
ALTER TABLE public.profile_photos ENABLE ROW LEVEL SECURITY;
REVOKE ALL ON public.profile_photos FROM PUBLIC, anon, authenticated;
```

20261008193359_account_invitations.sql

```
-- School authorization remains in users/account_roles/guardian_links.
-- These private tables are used only by the authenticated Flask backend.
CREATE TABLE public.account_identities (
  username TEXT PRIMARY KEY REFERENCES public.users(username) ON DELETE CASCADE,
  email TEXT NOT NULL UNIQUE CHECK (email=lower(email) AND length(email)<=254),
  verified_at BIGINT
);
CREATE TABLE public.account_invitations (
  id TEXT PRIMARY KEY,
  username TEXT NOT NULL UNIQUE REFERENCES public.users(username),
  email TEXT NOT NULL,
  token_hash TEXT UNIQUE,
  expires_at BIGINT,
  status TEXT NOT NULL DEFAULT 'draft' CHECK(status IN
('draft','sending','sent','failed','used','revoked')),
  created_by TEXT NOT NULL REFERENCES public.users(username),
  created_at BIGINT NOT NULL,
  last_sent_at BIGINT,
  used_at BIGINT
```

```
);
CREATE INDEX account_invitations_creator_idx ON public.account_invitations(created_by);
CREATE TABLE public.enrollment_settings (key TEXT PRIMARY KEY,value TEXT NOT NULL);
CREATE TABLE public.auth_rate_limits (
  key TEXT NOT NULL, bucket_window BIGINT NOT NULL, attempts INTEGER NOT NULL CHECK(attempts>0),
  PRIMARY KEY(key,bucket_window)
);
CREATE INDEX auth_rate_limits_window_idx ON public.auth_rate_limits(bucket_window);
ALTER TABLE public.account_identities ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.account_invitations ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.enrollment_settings ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.auth_rate_limits ENABLE ROW LEVEL SECURITY;
REVOKE ALL ON
public.account_identities,public.account_invitations,public.enrollment_settings,public.auth_rate_limits
FROM PUBLIC,anon,authenticated;
```

20261015212136_configurable_school_schedule.sql

```
-- Additive: retain existing slots and the closed academic history.
alter table public.schedule_slots add column if not exists active boolean not null default true;
alter table public.schedule_slots add column if not exists revision integer not null default 1;
-- Existing RLS and private-server-only grants remain unchanged.
```

20261016200538_lesson_attendance_and_mfa.sql

```
-- The runtime uses Python weekday: Monday=0, Sunday=6.
-- Only an empty legacy schedule can be adopted automatically; populated
-- installations need an explicit 1-based -> 0-based data migration first.
DO $$ BEGIN
  IF EXISTS (SELECT 1 FROM public.schedule_slots) THEN
    RAISE EXCEPTION 'Review existing schedule weekday values before adopting Python weekday';
  END IF;
END $$;
ALTER TABLE public.schedule_slots DROP CONSTRAINT schedule_slots_weekday_check;
ALTER TABLE public.schedule_slots ADD CONSTRAINT schedule_slots_weekday_check CHECK (weekday BETWEEN 0
AND 6);

ALTER TABLE public.lessons ADD COLUMN IF NOT EXISTS attendance_revision integer NOT NULL DEFAULT 0;
CREATE UNIQUE INDEX IF NOT EXISTS lessons_subject_start_idx ON
public.lessons(class_subject_id,starts_at);
CREATE INDEX IF NOT EXISTS attendance_enrollment_idx ON
public.attendance_records(enrollment_id,lesson_id);

CREATE TABLE public.account_mfa (
  username text PRIMARY KEY REFERENCES public.users(username),secret_ciphertext text,
  pending_ciphertext text,pending_until bigint,enabled boolean NOT NULL DEFAULT false,
  last_counter bigint NOT NULL DEFAULT -1
);
CREATE TABLE public.mfa_recovery_codes (
  username text NOT NULL REFERENCES public.account_mfa(username),code_hash text NOT NULL,
  used_at timestamptz,PRIMARY KEY(username,code_hash)
);
ALTER TABLE public.account_mfa ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.mfa_recovery_codes ENABLE ROW LEVEL SECURITY;
REVOKE ALL ON public.account_mfa,public.mfa_recovery_codes FROM anon,authenticated,PUBLIC;
```

20261020004001_faenet_identity_bridge.sql

```
BEGIN;
CREATE TABLE public.faenet_subjects (
  username text PRIMARY KEY REFERENCES public.users(username),
  subject text NOT NULL UNIQUE
);
CREATE TABLE public.faenet_grants (
  code_hash text PRIMARY KEY,
  username text NOT NULL REFERENCES public.users(username),
  auth_version bigint NOT NULL,
  challenge text NOT NULL,
  expires_at bigint NOT NULL,
  token_hash text UNIQUE,
  token_expires_at bigint
);
ALTER TABLE public.faenet_subjects ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.faenet_grants ENABLE ROW LEVEL SECURITY;
REVOKE ALL ON TABLE public.faenet_subjects,public.faenet_grants FROM PUBLIC,anon,authenticated;
COMMIT;
```

6. MAPA DE NAVEGAÇÃO

Entrada → identificação → validação → segunda confirmação, quando ativa → painel do perfil.

Perfil	Caminhos a partir do painel
Aluno	Boletim, Agenda, Horário, Exercícios, Comunicados, Secretaria, Estágios e Mensagens.
Professor	Minhas turmas, Notas, Chamada, Diário, Exercícios, Avisos, Estágios, Mensagens e Calendário.
Direção	Usuários, Estrutura, Secretaria, Coordenação, Períodos, Calendário, Relatórios e Configurações.
Responsável	Visão geral, estudante vinculado, Secretaria, Calendário, Notificações e Mensagens.
Menu da conta	Foto de perfil, segurança e saída. FaeNet abre o fluxo integrado quando disponível.

7. MANUAL DO USUÁRIO

7.1. Acesso ao sistema

Entrar e sair do campus

A tela inicial identifica o perfil e valida as credenciais antes de abrir o espaço correspondente. E-mail confirmado ou nome de usuário podem identificar uma conta elegível.

1. Abra o endereço público do FaeHub+ e escolha Aluno, Professor, Diretor ou Responsável.

2. Informe seu e-mail ou usuário e sua senha. Clique em Validar acesso.

3. Se a conta tiver autenticador ativo, conclua a segunda confirmação com o método oferecido.

4. Na confirmação de identidade, escolha Entrar no campus. Para encerrar, abra seu nome no cabeçalho e use Sair da conta.

Ativar uma conta por convite

O cadastro é autorizado pela escola. Não existe inscrição pública que permita escolher livremente um papel escolar. O titular confirma o e-mail e cria sua própria senha pelo convite recebido.

1. Peça à direção o cadastro com seu e-mail correto e os vínculos acadêmicos necessários.

2. Abra o link do convite na sua própria caixa de e-mail. A tela de Primeiro acesso sem convite não libera a continuação.

3. Confira a identidade apresentada, escolha uma senha entre 8 e 128 caracteres e repita a mesma senha.

4. Conclua a confirmação do e-mail e a ativação. Depois retorne ao login com as credenciais pessoais.

Recuperar senha ou nome de usuário

A recuperação utiliza apenas o e-mail confirmado da conta e não depende de aprovação da direção. A resposta pública é neutra para não revelar se um endereço está cadastrado.

1. No login, clique em Recuperar senha ou usuário e informe o e-mail confirmado.

2. Escolha Meu usuário para receber sua identificação por e-mail, ou Minha senha para receber um código de uso único.

3. Para redefinir, informe o mesmo e-mail e o código mais recente, válido por 30 minutos. Digite e confirme a nova senha.

4. Conclua a redefinição e entre novamente. Confira o spam se a mensagem não aparecer na caixa de entrada.

Utilizar o autenticador da conta

Segurança da conta permite adicionar TOTP, compatível com Google Authenticator e outros aplicativos. É um segundo fator: o Gmail envia mensagens, enquanto o autenticador gera códigos no celular. São serviços diferentes.

1. Abra seu nome no cabeçalho e selecione Segurança da conta. Confirme que o servidor está configurado e que sua conta foi ativada por e-mail.

2. Confirme a senha atual, leia o QR no autenticador ou cadastre a chave manualmente e confirme um código de seis dígitos antes da expiração de dez minutos.

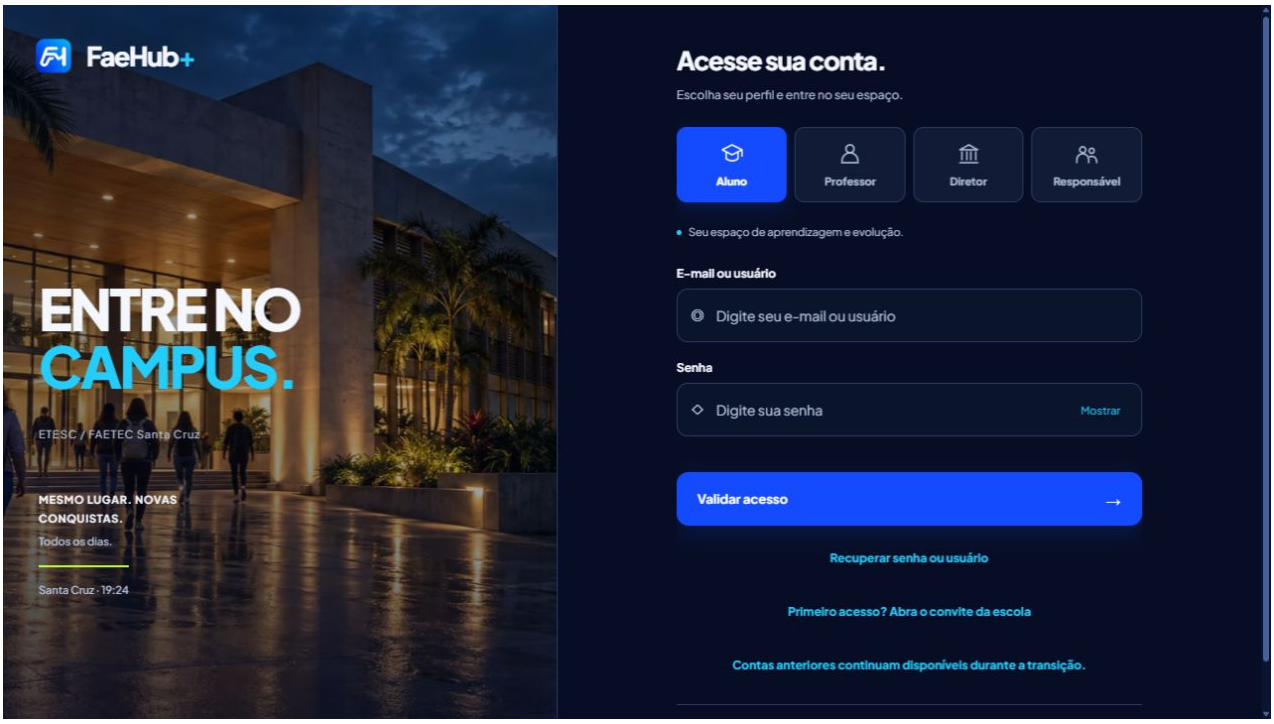
3. Guarde os dez códigos de recuperação em local privado. Eles são exibidos uma vez e cada um pode ser usado apenas uma vez, junto com a senha.

4. Para gerar novos códigos ou desativar o recurso, use a opção correspondente e confirme senha e um código novo. Nunca envie QR ou códigos por mensagens.

7.2. Apresentação das telas e funcionalidades

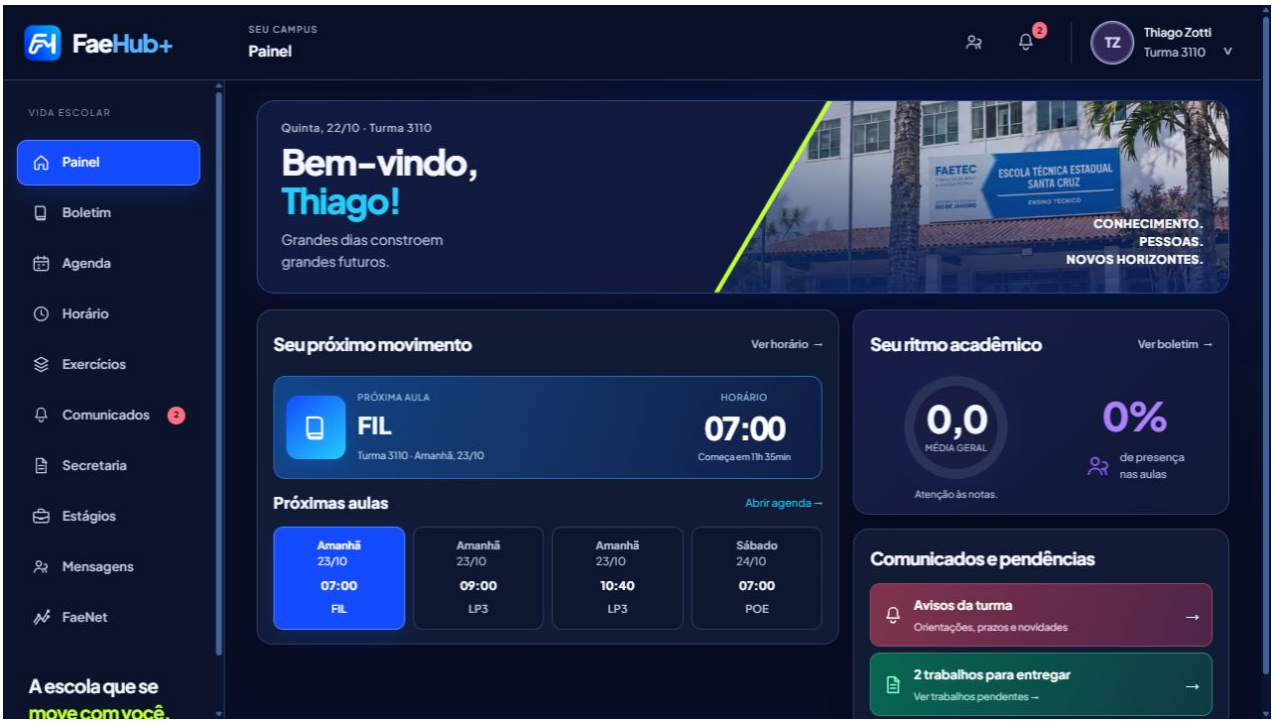
As capturas mostram a interface com dados de demonstração. Os itens disponíveis dependem do perfil e dos vínculos da conta.

Tela de entrada



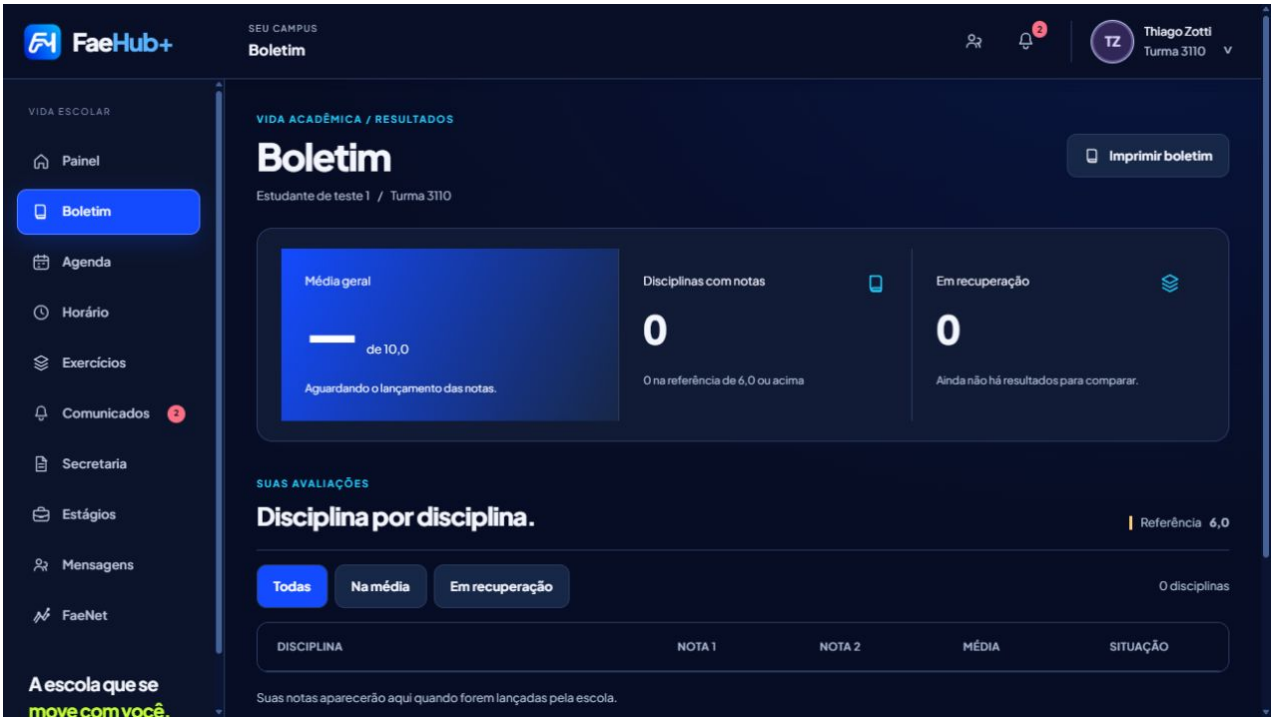
Captura em ambiente de demonstração do FaeHub+.

Painel do aluno



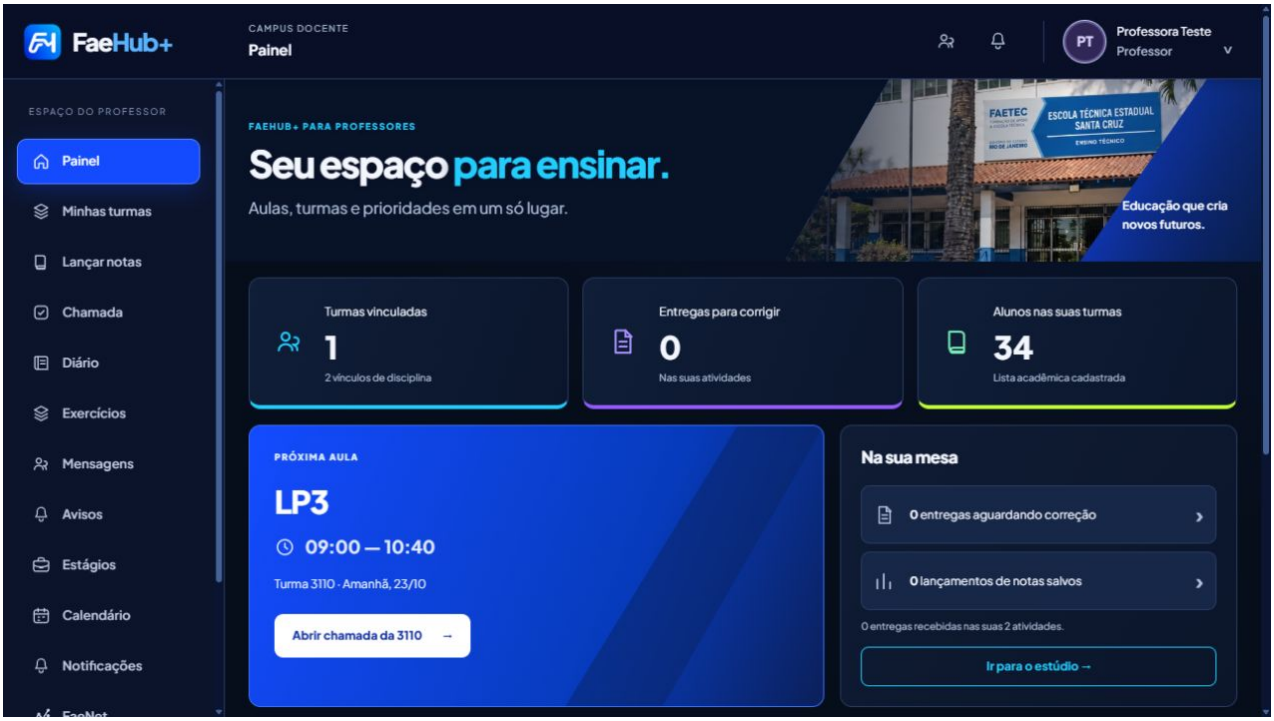
Captura em ambiente de demonstração do FaeHub+.

Boletim



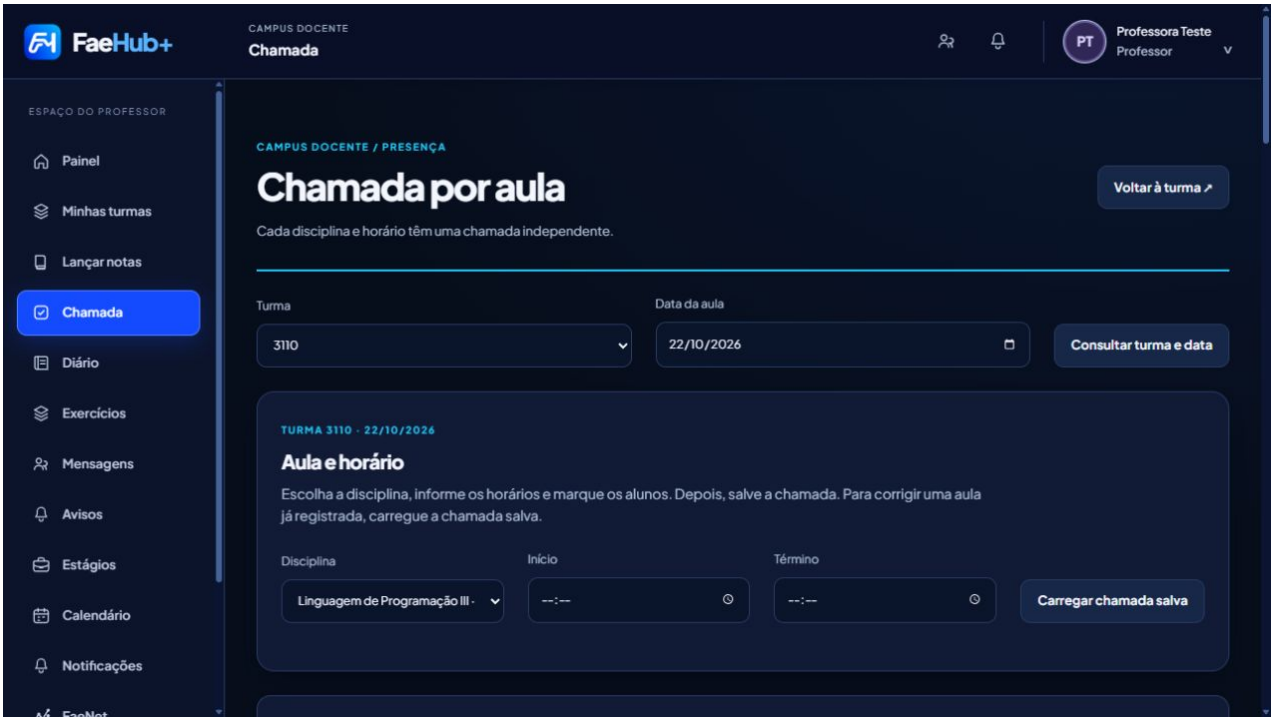
Captura em ambiente de demonstração do FaeHub+.

Painel do professor



Captura em ambiente de demonstração do FaeHub+.

Chamada por aula



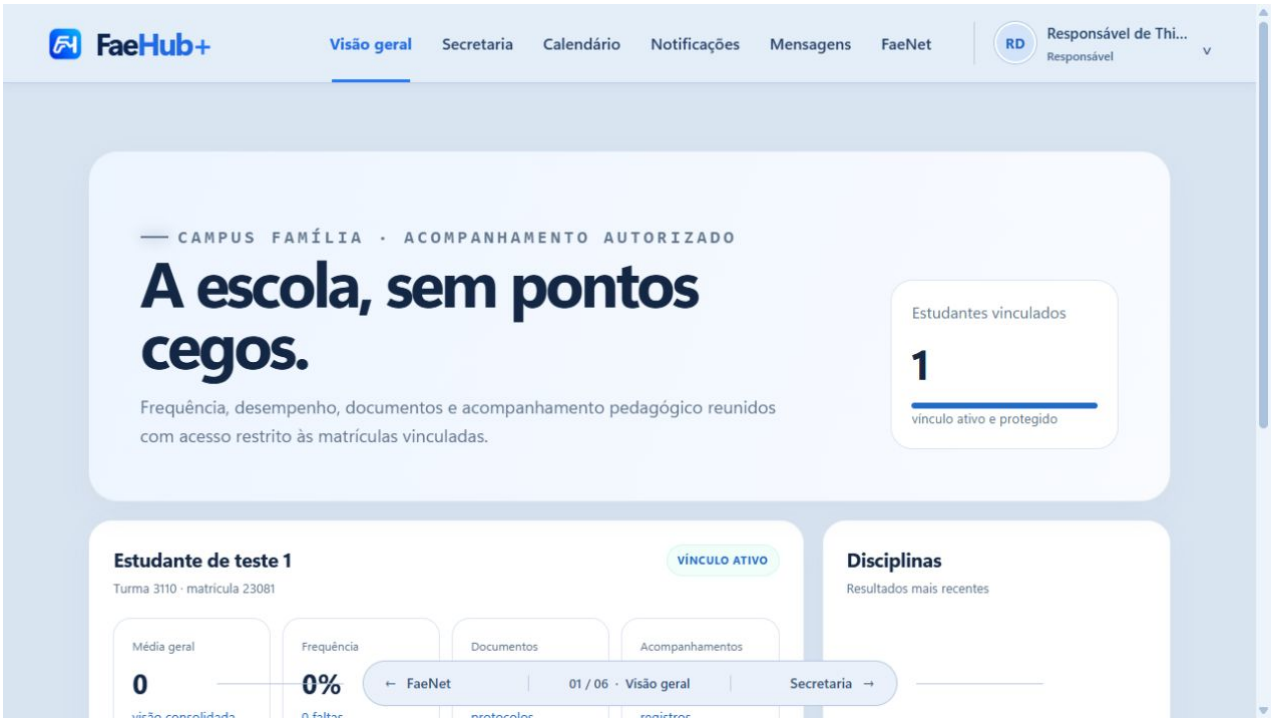
Captura em ambiente de demonstração do FaeHub+.

Estrutura acadêmica



Captura em ambiente de demonstração do FaeHub+.

Visão do responsável



Captura em ambiente de demonstração do FaeHub+.

7.3. Instruções de utilização

Manual do aluno

Consultar o painel do aluno

O painel resume a turma, a próxima aula, as aulas seguintes, o ritmo acadêmico e pendências. Os cartões são atalhos para as áreas de consulta e entrega, não substitutos do histórico detalhado.

1. Confira a identificação e a turma mostradas no início do painel.

2. Use Ver horário ou Abrir agenda para conferir disciplina, dia e início das próximas aulas.

3. Abra Ver boletim para analisar os lançamentos de notas. Confira a frequência na Agenda e seu histórico de chamadas.

4. Use Avisos da turma ou o cartão de trabalhos pendentes para abrir a lista correspondente.

Ler e imprimir o Boletim

O Boletim apresenta Nota 1, Nota 2, média e situação por disciplina. O resumo reúne a média dos lançamentos disponíveis, a quantidade de disciplinas com notas e as que estão em recuperação.

1. Abra Boletim e confirme o estudante e a turma. Na tabela, compare N1, N2 e a média de cada disciplina.

2. Use Todas, Na média ou Em recuperação para filtrar. A referência exibida é 6,0 e a média da disciplina é (N1 + N2) / 2.

3. Abra Simular uma média para experimentar valores. A simulação não altera o boletim oficial nem grava notas.

4. Use Imprimir boletim para abrir a impressão do navegador e escolher uma impressora ou Salvar como PDF.

Acompanhar Agenda e frequência

A Agenda reúne o calendário de presenças, os eventos visíveis para a turma e as chamadas do mês, com disciplina e horário. O histórico diário anterior fica separado da nova frequência por aula.

1. Abra Agenda, escolha o mês em Consultar mês e clique em Atualizar.

2. Selecione um dia para consultar o registro; confira as chamadas por disciplina e horário na lista do mês.

3. Leia os próximos eventos e observe o público, a data e o local publicados.

4. Se precisar de registros antigos, abra Histórico diário anterior. Use Consultar por aula para retornar à base atual.

Interpretar a grade de horários

A grade organiza as aulas publicadas por dia, início e término. Exibe também períodos sem aula cadastrada e identifica o docente quando há vínculo correspondente.

1. Abra Horário e confira a turma, o período abrangido e a próxima aula na grade.

2. Escolha Semana completa ou um dia específico para reduzir a visualização.

3. Leia cada bloco com início, término, disciplina e docente. Diferencie blocos de aula de Sem aula ou Tempo livre.

4. Use Preparar minhas entregas para ir a Exercícios. Havendo divergência na grade, informe a secretaria.

Buscar atividades e consultar devolutivas

Exercícios reúne atividades ativas da turma, prazo, enunciado, anexos e situação da entrega. A busca e os filtros ajudam a separar o que ainda precisa ser feito do que já foi enviado.

1. Abra Exercícios e procure pelo título. Use Todas, Pendentes ou Entregues para restringir a lista.

2. Clique em Abrir atividade para ler o enunciado completo, os materiais e a data de entrega.

3. Depois do envio, retorne à atividade para conferir sua resposta, o anexo e uma eventual correção do professor.

4. Se o professor solicitar ajuste, revise a resposta com cuidado e confirme o novo envio pelo formulário.

Entregar uma atividade com anexo

O formulário permite responder por texto, informar o link do trabalho ou anexar um arquivo. A entrega só é gravada após a ação explícita de envio.

1. Abra a atividade correta e leia suas instruções antes de preencher a resposta.

2. Digite o texto ou link solicitado. Para anexar, clique em Adicionar arquivo e escolha um arquivo dentro dos formatos e do tamanho permitidos.

3. Confira o nome do arquivo selecionado. Não envie documentos pessoais desnecessários ou links sem permissão de leitura para o professor.

4. Clique em Entregar atividade e confira a confirmação e a situação de entregue. Reabra para verificar o que ficou salvo.

Ler e organizar os Comunicados

A central separa Avisos da turma, publicados por professores, de Atualizações da conta, geradas por serviços do sistema. A leitura fica persistida por conta.

1. Abra Comunicados e leia o conteúdo de cada aviso ou atualização.

2. Use Marcar como lido ou Marcar como lida para registrar a leitura de um item.

3. Para registrar a leitura dos itens elegíveis de uma vez, use Marcar tudo como lido.

4. Nas atualizações, Abrir contexto leva ao documento, evento ou área relacionada. Quando disponível, o descarte de aviso pode ser desfeito.

Solicitar e acompanhar documentos

A Secretaria digital registra pedidos por matrícula e disponibiliza um protocolo de acompanhamento. O aluno consulta a própria matrícula; o responsável seleciona uma das matrículas vinculadas.

1. Abra Secretaria, confira a matrícula e selecione o tipo de documento.

2. Informe a finalidade e clique em Gerar protocolo. Anote o protocolo apresentado.

3. Acompanhe a situação e as observações da escola em Acompanhamento.

4. Quando a situação permitir, clique em Abrir. No documento emitido, use Imprimir / salvar PDF e confira a identificação do estudante.

Consultar oportunidades de estágio

Estágios apresenta oportunidades divulgadas pelos professores para a turma. O app funciona como mural de anúncios e não como sistema de contratação ou garantia de vaga.

1. Abra Estágios e use a busca por oportunidade.

2. Filtre por Todas, Presencial, Híbrido ou Remoto. Confira o prazo de inscrição.

3. Abra o anúncio para ler empresa, descrição, requisitos, local, carga horária e instruções.

4. Inscreva-se pelo link ou canal indicado pelo anúncio, depois de verificar a confiabilidade do destino.

Manual do professor

Organizar o trabalho pelo painel docente

O painel docente resume turmas vinculadas, disciplinas, alunos, entregas a corrigir e próximas aulas. Os atalhos conduzem às ações de registro e publicação.

1. Confira os totais de turmas e vínculos, sem confundir número de alunos com quantidade de contas de login.

2. Em Próxima aula, use Abrir chamada para iniciar o registro da turma correspondente.

3. Consulte Na sua mesa para abrir as entregas a corrigir e os lançamentos já salvos.

4. Use os atalhos Lançar notas, Registrar frequência e Criar atividade. Em Minhas turmas, confirme os componentes atribuídos.

Consultar as turmas vinculadas

Minhas turmas organiza a lista de estudantes e as disciplinas atribuídas à conta docente. A busca auxilia a localizar um estudante antes de registrar notas ou presença.

1. Abra Minhas turmas e selecione a turma disponível.

2. Confira a quantidade de alunos e os componentes curriculares vinculados.

3. Use Buscar aluno para localizar nome ou matrícula na lista.

4. Use Abrir chamada, Lançar notas ou o atalho Notas do estudante para abrir o contexto apropriado.

Lançar e corrigir notas

A tela seleciona turma e disciplina e permite salvar N1 e N2 para cada aluno. A média durante a edição é uma prévia; o botão de uma linha salva somente aquele estudante e disciplina.

1. Selecione a turma e a disciplina e clique em Abrir boletins.

2. Localize o aluno pela busca e confira sua matrícula.

3. Preencha N1 e N2, confira a média prevista e clique em Salvar na mesma linha.

4. Após a confirmação, consulte o registro novamente. Repita a ação para outros estudantes; não presuma que salvar uma linha gravou todas.

Realizar a chamada por aula

Cada disciplina e início de aula têm uma chamada independente. O dia sozinho não identifica o registro. Alterar a seleção de alunos não grava dados até usar Salvar chamada desta aula.

1. Consulte turma e data. Selecione a disciplina e informe início e término da aula no horário da escola.

2. Para corrigir uma chamada existente, use Carregar chamada salva antes de editar.

3. Marque todos os estudantes. O atalho Marcar os alunos como presentes preenche a turma; depois marque as exceções como Ausente. A busca não reduz a obrigação de conferir a turma inteira.

4. Revise os contadores de presentes, ausentes e sem marcação. Clique em Salvar chamada desta aula e confirme os registros ao recarregar a mesma seleção.

Registrar e enviar o Diário

O Diário registra o conteúdo pedagógico de uma aula e permite guardar rascunho ou enviá-lo à validação. Ele não substitui a chamada de presença.

1. Selecione turma, disciplina e data; informe os tempos de aula.

2. Descreva conteúdo e objetivos e complete os demais campos pedagógicos necessários.

3. Use Salvar rascunho se o registro ainda estiver em elaboração, ou Enviar para validação quando estiver concluído.

4. Acompanhe Meus registros. Caso seja devolvido com ajustes, leia a observação, corrija o registro permitido e envie novamente.

Publicar atividades e corrigir entregas

O estúdio permite criar, editar, arquivar, restaurar e consultar entregas das próprias atividades. O arquivamento preserva o histórico, em vez de apagar respostas de alunos.

1. Use Criar atividade e selecione uma turma e disciplina vinculadas. Informe título, prazo e instruções; anexe material se necessário.

2. Clique em Publicar para os alunos. Depois, utilize a busca e os filtros Ativos, Arquivados ou Todos.

3. Abra Ver entregas e expanda o aluno. Leia resposta e arquivos, atribua a nota da atividade, escreva feedback e salve a correção.

4. Para retirar a atividade da lista ativa, use Arquivar. No filtro Arquivados, a ação de restauração permite reativá-la.

Publicar e arquivar avisos da turma

Avisos permite manter o mural da turma sem misturar publicações docentes com notificações administrativas ou conversas individuais.

1. Abra Avisos e clique em Novo comunicado.

2. Selecione a turma vinculada, escreva título e conteúdo e defina a prioridade.

3. Publique e confira a mensagem na lista de ativos. Use Editar para revisar uma publicação sua.

4. Arquive um comunicado que não deva continuar ativo. Use o filtro Arquivados e a restauração para disponibilizá-lo novamente.

Publicar oportunidades profissionais

A publicação de oportunidade associa um anúncio à turma e define como o aluno pode buscar informações ou se inscrever.

1. Clique em Publicar oportunidade e selecione uma turma vinculada.

2. Preencha os dados da empresa e da vaga, o prazo e a modalidade Presencial, Híbrido ou Remoto.

3. Informe um link seguro ou instruções de inscrição e publique para os alunos.

4. Use busca e filtros de modalidade e visibilidade. Edite, arquive ou restaure suas publicações quando houver mudança de prazo ou disponibilidade.

Manual da direção

Acompanhar o observatório da direção

O painel administrativo apresenta alunos cadastrados, contas ativas, professores, turmas, filas de atendimento, calendário e vínculos da equipe.

1. Confira os totais e diferencie cadastro acadêmico, conta de acesso e vínculo docente.

2. Na Fila de atendimento, abra diários a validar, documentos em andamento ou acompanhamentos abertos.

3. Consulte calendário e panorama das turmas para localizar o contexto da atividade escolar.

4. Use Gerenciar comunidade para Usuários ou Ver relatórios para exportações e trilha de atividade.

Gerenciar contas e convites

Usuários organiza identidades e acesso, com busca, filtro por perfil, preparação de convites e gerenciamento das contas. O cadastro acadêmico de um estudante é mantido na Estrutura.

1. Confirme que sua conta de direção está ativada e que o envio de e-mail está configurado.

2. Use Preparar convite para cadastrar a identificação e o perfil correto, com e-mail do titular e vínculos exigidos pelo formulário.

3. Em Gerenciar, confira os dados da conta antes de aplicar alterações, reenviar ou revogar um convite ou alterar a situação permitida.

4. Utilize busca e filtro de perfil para conferir a comunidade. A ação Encerrar contas de demonstração só deve ser usada após garantir o acesso pessoal da direção.

Organizar a estrutura acadêmica

Estrutura conecta anos e períodos, cursos, disciplinas, turmas, matrículas, professores e horários. O Mapa mostra o inventário e a sequência de dependências dos cadastros.

1. Abra Estrutura e consulte o Mapa para conhecer os cadastros já existentes.

2. Use Anos e períodos para criar o ciclo letivo, delimitar suas datas e cadastrar os períodos dentro desse ano.

3. Defina cursos e disciplinas, depois crie turmas e associe seus componentes.

4. Cadastre estudantes e matrículas e, por fim, os vínculos docentes e os horários. Use Consultar históricos para recuperar uma trajetória anterior.

Cadastrar anos e períodos letivos

A área apresenta a linha do tempo dos anos e seus períodos. Cada ciclo tem datas e situação próprias, preservando os anteriores.

1. Em Novo ano letivo, informe identificação e intervalo de datas e use Criar ano.

2. Abra Novo período e associe-o ao ano correto, com ordem e datas coerentes.

3. Confira na linha do tempo os períodos e a situação do ano antes de atualizar.

4. Para um fechamento ou reabertura operacional, use Períodos e registre uma justificativa auditável.

Manter cursos e disciplinas

O catálogo pedagógico mantém cursos e disciplinas reutilizáveis entre anos. Uma disciplina só passa a compor uma turma após o vínculo na grade dessa turma.

1. Consulte os códigos e nomes existentes para evitar duplicação.

2. Use Novo curso ou Nova disciplina, preencha os campos e confirme a criação.

3. Vincule a disciplina à turma em Turmas e grade; o simples cadastro no catálogo não faz essa associação.

4. Para deixar de utilizar um item, use a ação de desativação ou ativação correspondente, preservando os vínculos históricos.

Cadastrar turmas e compor grades

Cada turma pertence a um ano letivo e pode estar associada a um curso. A grade curricular define as disciplinas oferecidas nessa turma; os horários semanais são publicados em uma área separada.

1. Em Nova turma, selecione ano e curso e informe identificação, etapa, turno, sala e capacidade.

2. Crie a turma e confira o registro no inventário.

3. Na turma correta, escolha uma disciplina do catálogo e clique em Vincular. Repita para os demais componentes.

4. Salve alterações de situação apenas após verificar matrículas e operação do ano. Depois configure os vínculos docentes e horários.

Cadastrar estudantes e matrículas

Estudante é a pessoa no cadastro acadêmico; matrícula relaciona essa pessoa a uma turma e a um período de vínculo. Conta de acesso é outro registro, que pode ser associado à identificação escolar.

1. Pesquise por nome, matrícula, turma ou ano e percorra a paginação antes de criar um novo registro.

2. Cadastre o estudante quando ele ainda não existir no diretório.

3. Abra Nova matrícula e relacione o estudante à turma correta, com a data adequada.

4. Quando necessário, utilize Encerrar e informe o motivo ou situação de saída oferecida. Consulte o histórico para conferir a trajetória preservada.

Definir vínculos docentes

O vínculo docente conecta professor, turma e disciplina, com vigência. Essa relação determina quais registros o professor pode consultar ou lançar.

1. Confira a conta ativa do professor e a oferta da disciplina na turma.

2. Selecione professor e combinação de turma e disciplina e informe a data de início.

3. Clique em Criar vínculo e confira o resultado no Mapa docente.

4. Quando houver mudança de responsabilidade, encerre o vínculo anterior e cadastre o novo respeitando as datas escolares.

Publicar e revisar horários semanais

Horários associa uma disciplina oferecida a dia, início, término e sala. A grade publicada alimenta as consultas e os próximos horários nos painéis.

1. Selecione Turma e ano e use Consultar grade para visualizar os horários existentes.

2. Em Cadastrar horário, selecione a disciplina e informe dia, início, término e sala.

3. Clique em Salvar horário e confira a linha publicada. Use Editar para ajustar um registro existente.

4. Quando um horário deixar de vigorar, utilize Arquivar. Confira a visualização do aluno e os vínculos docentes depois da manutenção.

Atender e emitir documentos

A central de protocolos permite localizar solicitações e registrar o andamento do atendimento. O titular acompanha as atualizações na sua própria Secretaria e nos comunicados elegíveis.

1. Localize o pedido na Central de protocolos e confira matrícula, tipo e finalidade.

2. Clique em Atualizar, escolha Em processamento, Pronto, Entregue ou Indeferido e escreva uma observação útil.

3. Use Salvar andamento e confira o estado exibido na lista.

4. Quando disponível, use Abrir para conferir o documento e Imprimir / salvar PDF. Marque Entregue somente após concluir a entrega pelo procedimento da escola.

Registrar acompanhamento pedagógico

Coordenação reúne alertas derivados de rendimento e frequência e permite registrar intervenções com plano de ação. O registro deve manter contexto objetivo e acesso restrito.

1. Consulte o Radar acadêmico e confira os registros do estudante antes de interpretar um alerta.

2. Em Novo acompanhamento, selecione o estudante e descreva categoria, ocorrência pedagógica e plano de ação.

3. Use Abrir acompanhamento e confira a entrada na lista de planos.

4. Registre as atualizações permitidas e acompanhe as situações Aberto, Em acompanhamento ou Fechado. Utilize Validar diários para acessar a fila pedagógica.

Fechar e reabrir períodos

Períodos controla a proteção dos lançamentos após fechamento. Reabrir não cria um novo período nem elimina registros: altera o estado do período existente e registra uma justificativa.

1. Confira nome, ano e datas do período antes de qualquer alteração.

2. Use Fechar em um período aberto ou Reabrir em um período fechado.

3. Informe o motivo da alteração, entre 5 e 500 caracteres, no fluxo apresentado e confirme.

4. Confira a situação atual e a Trilha de auditoria, com ação, responsável, data e motivo.

Publicar eventos no Calendário

O Calendário mantém eventos, feriados, recessos, avaliações, reuniões e conselhos, direcionados à comunidade ou a um público específico.

1. Consulte a Linha do tempo para evitar publicação repetida ou datas conflitantes.

2. Preencha título e intervalo de data e hora, selecione o tipo e defina o público correto.

3. Para público de turma, informe a identificação da turma. Complete local e descrição e use Publicar no calendário.

4. Confira a visibilidade no perfil correspondente. Se o evento deixar de ocorrer, utilize Cancelar evento no registro adequado.

Consultar relatórios e exportar registros

Relatórios apresenta contadores operacionais, recorte de presença, notas, exportações CSV e trilha de atividade. Os arquivos podem conter dados pessoais e acadêmicos protegidos.

1. Consulte as filas de diários, documentos, acompanhamentos e eventos futuros.

2. Para presença, informe as datas inicial e final e use Aplicar período. Limpar remove o recorte.

3. Escolha a exportação desejada: Diário de classe, Documentos, Acompanhamentos, Presenças do período ou Notas registradas.

4. Confira a base do percentual, os registros recentes e a Trilha de atividade. Abra o CSV em uma ferramenta compatível sem compartilhar o arquivo publicamente.

Manter os dados institucionais

Configurações mantém informações institucionais exibidas na central da direção. Essa tela não administra credenciais técnicas de Gmail, Drive, banco ou autenticador.

1. Confira os valores atuais antes de editar.

2. Informe o nome da unidade, o ano de referência e o contato institucional corretos.

3. Clique em Salvar configurações e confira os valores após a confirmação.

4. Para usuários, vínculos, turmas ou períodos, abra as áreas correspondentes em vez de tentar controlar tudo pelo ano de referência.

Consultar a trajetória acadêmica

O histórico organiza anos, turmas, curso, matrícula e resultados preservados. Ele permite acompanhar uma trajetória sem sobrescrever ciclos anteriores ao abrir um novo ano.

1. Na direção, abra Consultar históricos pelo Mapa da Estrutura ou Abrir consulta pela Secretaria.

2. Selecione o estudante autorizado e confira a identificação e o ano.

3. Leia a turma, a situação da matrícula e os resultados disponíveis para cada ciclo.

4. Para documento formal, solicite ou emita o tipo adequado pela Secretaria; a consulta visual não substitui a emissão.

Manual do responsável

Acompanhar estudantes vinculados

O Portal do responsável mostra os estudantes vinculados, sua turma, média consolidada, frequência, documentos e quantidade de acompanhamentos. Não libera a base escolar completa.

1. Entre pelo perfil Responsável com sua própria conta e confira a lista de estudantes vinculados.

2. Leia os resultados e a frequência do estudante correto, considerando se existem lançamentos e chamadas disponíveis.

3. Use Solicitar documento para abrir a Secretaria, Ver calendário para os eventos autorizados e Falar com a escola para Mensagens.

4. Se não houver matrícula vinculada, procure a escola para corrigir o cadastro. Não utilize as credenciais do estudante para contornar o vínculo.

Uso compartilhado

Navegar e personalizar a identificação

O menu lateral apresenta as áreas autorizadas. O cabeçalho reúne mensagens, comunicados ou notificações e o menu pessoal. A opção Pular para o conteúdo auxilia a navegação pelo teclado.

1. Escolha a seção desejada no menu. O título no cabeçalho informa onde você está.

2. Clique no nome da conta para encontrar Segurança da conta, Alterar foto de perfil e Sair da conta, nas telas que utilizam esse cabeçalho.

3. Em Alterar foto de perfil, escolha uma imagem permitida e confirme o envio apresentado pela interface. A foto identifica a conta; não altera nome, papel nem matrícula.

4. Use os botões de busca e os filtros de cada seção. Se uma lista estiver vazia, limpe os filtros antes de concluir que faltam registros.

Conversar com pessoas e grupos

Mensagens reúne conversas individuais e grupos autorizados, com busca e envio de texto ou anexo. Conversas da FaeHub+ não são as mesmas conversas da FaeNet.

1. Abra Mensagens. Escolha uma conversa existente ou clique em Nova conversa.

2. Em uma nova conversa, selecione uma pessoa da lista permitida e escreva a mensagem. Para a turma, abra o grupo correspondente.

3. Use Adicionar foto ou arquivo ou Anexar para escolher uma imagem, PDF, TXT ou ZIP permitido. Confira a seleção e clique em Enviar.

4. Use a busca para reencontrar conversas. Abra a conversa para acompanhar as respostas e os indicadores de leitura disponibilizados.

Gerenciar notificações da conta

A central reúne atualizações acadêmicas e administrativas visíveis para a conta, com estado de leitura persistente. Para o aluno, a central de Comunicados reúne os avisos e as atualizações de forma integrada.

1. Abra Notificações no menu ou pelo sino do cabeçalho.

2. Use Todas, Não lidas e Lidas para consultar o histórico apresentado.

3. Abra o contexto de uma atualização quando houver um atalho para a área de origem.

4. Marque um item como lido ou utilize Marcar tudo como lido para os itens elegíveis da sua central.